

CS 6120

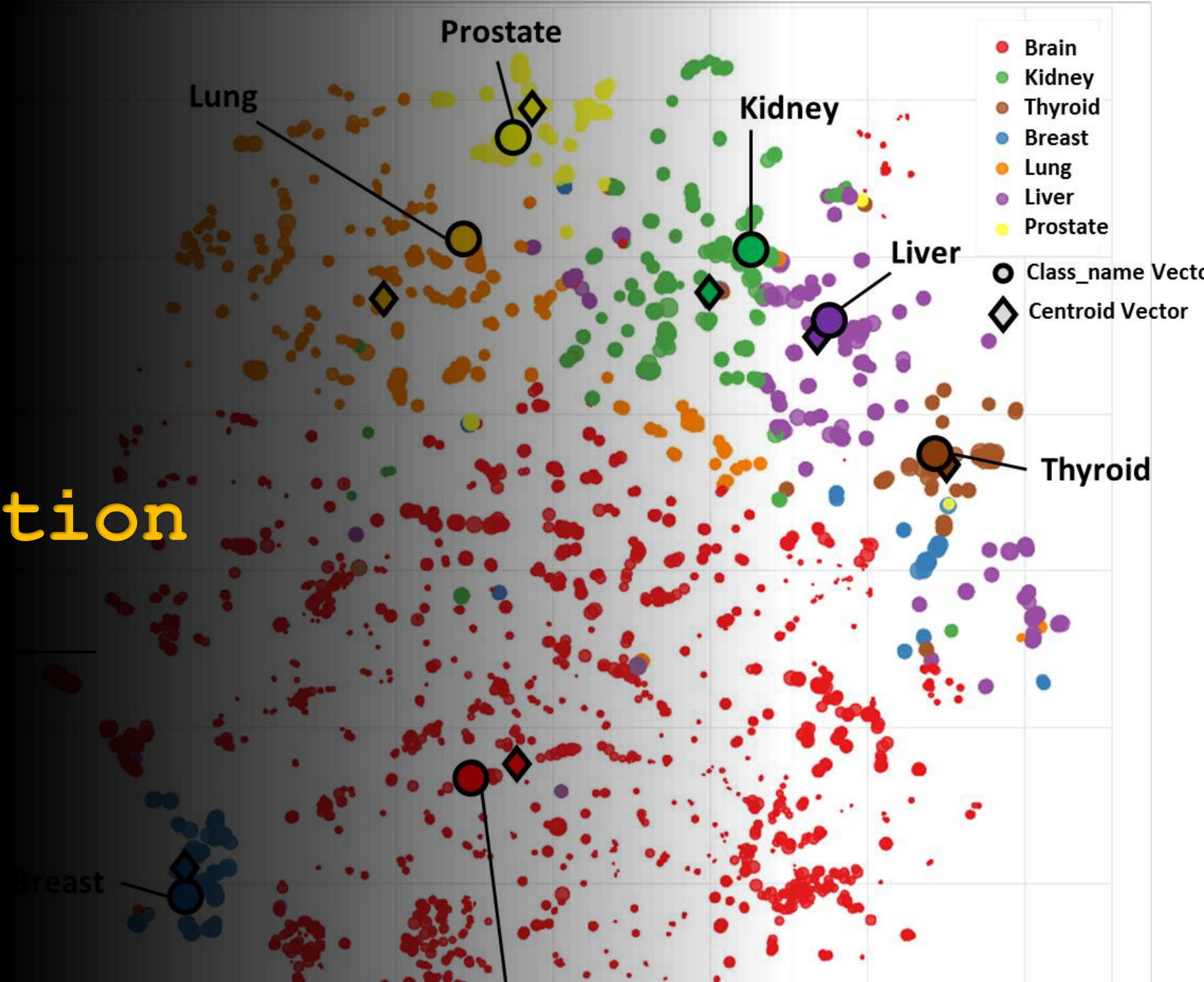
Natural Language Processing

Instructor: Amir Tahmasebi, PhD

Fall 2026



Word Representation



Announcements

- *HW3 will be posted today due by Oct 12, 11:59pm ET.*
- *Next Session: Lab 1,2*

Student Presentation

Beri, Raghavendra

Saravanan, Akshiya Vaibhavi

Satishkumar, Harshit

Koshtu, Hema Sravani

Motiwale, Kanad

A Comparison of PCFG Models

A Survey of Syntactic-Semantic Parsing Based on Constituent and Dependency Structures

Syntactic Parsing: A Survey

Three New Probabilistic Models for Dependency Parsing: An Exploration

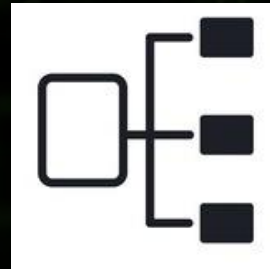
A Survey on Semantic Parsing

Syllabus

- What is Natural Language Processing?
 - History
 - Applications
- Python Primer
- Text Processing Fundamentals
 - Regular Expressions
 - Tokenization
 - Normalization
 - Libraries and Toolkits
- Parsing
 - Syntactic
 - Semantic
- Word Representation
 - TF-IDF
 - word2vec
 - Glove
- Intro to Machine Learning
 - Logistic Regression (LR)
 - Neural Networks
 - Recurrent Neural Networks (RNN)
 - Convolutional Neural Networks (CNN)
 - Transformers
- Language Models
- NLP Use Cases
 - Named Entity Recognition
 - Machine Translation
 - QA
- Large Language Models
 - chatGPT

Word Representation

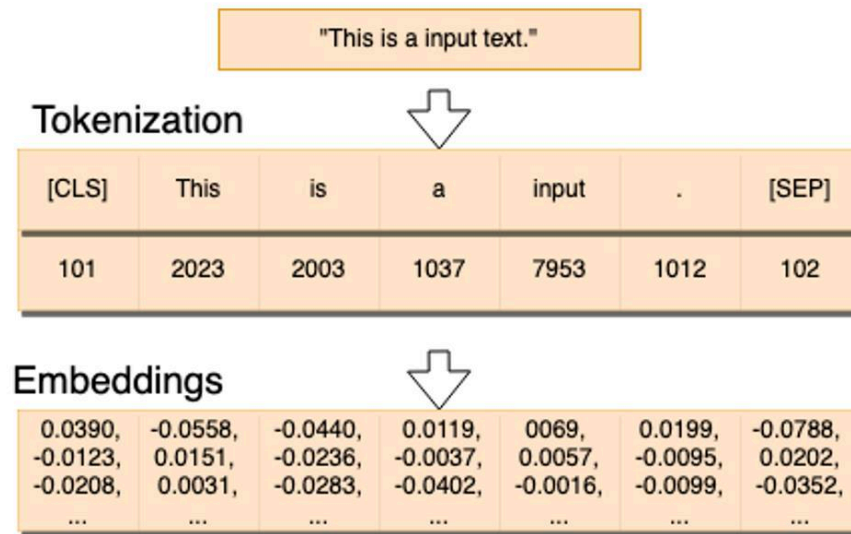
Machines understand numeric values



"Adenocarcinoma"
"Benign"
"Positive"
"Negative"

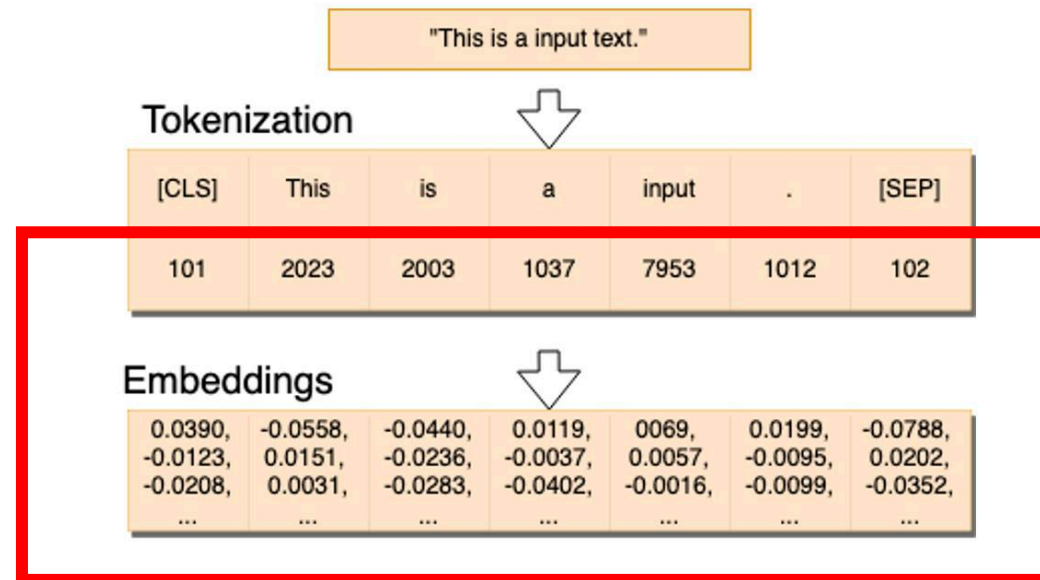
Word Representation

Tokenization



Word Representation

Tokenization



Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness:

Idea: Words with similar meanings should have similar representations.

It should capture the meanings of words, including nuances, connotations, and multiple senses of words. This allows the model to understand context and distinguish between different uses of the same word.

e.g. king and queen should have representations that are close to each other

Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness

Syntactic Awareness:

Idea: The representation should encode grammatical roles and behavior.

Beyond just meaning, an ideal representation also encodes syntactic roles and relationships, such as subject-verb agreement, object relations, and grammatical case, enabling the understanding of sentence structures.

run (verb) vs. *run* (noun)

- “I **run** every morning.”
- “He went for a **run**.”

Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness

Syntactic Awareness

Dimensionality and Sparsity:

Idea: Representations should be compact and dense, not huge and sparse.

While capturing a wide range of linguistic features, an ideal representation should also be efficient in terms of dimensionality, avoiding overly sparse representations that are computationally expensive and difficult to work with.

Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness

Syntactic Awareness

Dimensionality and Sparsity

Robustness to Noise:

Idea: Small variations or errors shouldn't break meaning.

An ideal word representation is robust to variations in input, such as typographical errors, slang, and informal or non-standard language use, ensuring reliable performance in real-world applications.

- *color* and *colour*
- *recieve* → *receive*

Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness

Syntactic Awareness

Dimensionality and Sparsity

Robustness to Noise

Learnability:

From sentences like:

“The **cat** sat on the mat.”

“The **dog** slept on the rug.”

The model learns that *cat* and *dog* behave similarly **without explicit supervision**, just from context.

Idea: Representations should be easy to learn from data.

It should be possible to learn the representation from data in an unsupervised or semi-supervised manner, without requiring extensive manual annotation or domain-specific tuning.

Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness

Syntactic Awareness

Dimensionality and Sparsity

Robustness to Noise

Learnability

Context Sensitivity:

- “I went to the **bank** to deposit money.”
- “The boat reached the **bank** of the river.”

Idea: The same word should have different representations in different contexts.

The representation should vary depending on the context in which a word is used, capturing how meaning changes with different linguistic surroundings. This is crucial for addressing polysemy and homonymy, where words have multiple meanings based on context.

Word Representation: *IDEAL*

Characteristics of an IDEAL Word Representation

Semantic Richness

Syntactic Awareness

Dimensionality and Sparsity

Robustness to Noise

Learnability

Context Sensitivity

Generalization Capability:

Idea: Representations should work for unseen or rare combinations.

It should allow for effective generalization from seen to unseen contexts or word combinations, enabling robust performance on NLP tasks even with limited data.

Word Representation: *Encode word with a unique number*

Integer Encoding

$$\{d_i\}_M, d_i \in \mathbb{N}$$

e.g. “There is a nodule in the left lung.”

“There”: d_1

“is”: d_2

“a”: d_3

...

Word Representation: *Encode word with a unique number*

Integer Encoding

$$\{d_i\}_M, d_i \in \mathbb{N}$$

Limitations

- does not capture any relationship between words
- challenging to interpret

Word Representation: *Encode word with a unique number*

Integer Encoding

$$\{d_i\}_M, d_i \in \mathbb{N}$$

Characteristic	Satisfied?	Explanation
Semantic Richness	✗ No	Integer values have no notion of meaning; <code>cat=12</code> and <code>dog=13</code> are not semantically closer than <code>cat=12</code> and <code>apple=103</code> .
Syntactic Awareness	✗ No	Grammatical roles (noun vs. verb) are not encoded; integers are arbitrary labels.
Dimensionality and Sparsity	⚠ Partial	Very low dimensional (1D), but <i>misleadingly dense</i> — numerical proximity has no semantic interpretation.
Robustness to Noise	✗ No	Typos or variants (e.g., <code>color</code> vs <code>colour</code>) map to completely unrelated integers.
Learnability	✗ No	Models may incorrectly infer ordinal relationships (e.g., <code>dog > cat</code>) that do not exist.
Context Sensitivity	✗ No	Each word has exactly one integer, regardless of context or meaning.
Generalization Capability	✗ No	No structure to support inference about unseen word combinations or relationships.

Word Representation: *character-based*

Integer Encoding (character-level)

$$\{d_i\}_M, d_i \in \mathbb{N}$$

e.g. “nodule”

“n”: d_1

”o”: d_2

”d”: d_3

...

Word Representation: *character-based*

Integer Encoding (character-level)

$$\{d_i\}_M, d_i \in \mathbb{N}$$

Limitations

- Words have variable lengths
- Sparsity

Word Representation: *character-based*

Integer Encoding (character-level)

$$\{d_i\}_M, d_i \in \mathbb{N}$$

Characteristic

Satisfied?

Explanation

Semantic Richness

✗ No

Individual characters carry no semantic meaning; $[3, 1, 20]$ does not encode that *cat* is an animal.

Syntactic Awareness

✗ No

Grammar operates at word/phrase level, not character level; characters do not encode POS or syntax.

Dimensionality and Sparsity

⚠ Partial

Low dimensional per character, but sequences can grow long and lack meaningful geometry.

Robustness to Noise

✓ Partial

Minor typos (*cat* → *kat*) change only one character, not the entire sequence.

Learnability

⚠ Partial

Models can learn patterns (prefixes, suffixes), but need **much more data** to infer meaning.

Context Sensitivity

✗ No

Each character has a fixed integer regardless of word or sentence context.

Generalization Capability

⚠ Partial

Can generalize to unseen words via spelling patterns, but not semantic relationships.

Word Representation: *One-hot Vector*

One-hot Vectors

$$\left\{ v \in \{0, 1\}^n : \sum_{i=1}^n v_i = 1 \right\}$$

v_k	0
	0
	.
	.
	.
	1
	.
	.
	0
	0

Word Representation: *One-hot Vector*

One-hot Vectors

e.g. “There is a nodule in the left lung.”

	a	...	in	...	is	...	left	...	lung	...	nodule	...	the	there	...
There	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
is	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
a	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
nodule	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
in			1	0	0	0	0	0	0	0	0	0	0	0	0
the	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
left	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
lung	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0

Word Representation: *One-hot Vector*

One-hot Vectors

$$\left\{ v \in \{0, 1\}^n : \sum_{i=1}^n v_i = 1 \right\}$$

Limitations

- Sparse representation
- Ignores context (surrounding words)
 - Words are intentionally dissimilar: dot product of two word vectors is 0.
- Inflexible to deal with new words due to fixed length

Word Representation: *One-hot Vector*

One-hot Vectors

$$\left\{ v \in \{0, 1\}^n : \sum_{i=1}^n v_i = 1 \right\}$$

Characteristic	Satisfied?	Explanation
Semantic Richness	✗ No	All words are orthogonal; <i>cat</i> and <i>dog</i> are no closer than <i>cat</i> and <i>banana</i> .
Syntactic Awareness	✗ No	No grammatical or structural information is encoded.
Dimensionality and Sparsity	✗ No	Very high-dimensional and extremely sparse (vocab size dimensions).
Robustness to Noise	✗ No	Any typo creates a completely new vector with no similarity to the original word.
Learnability	⚠ Partial	Linear models struggle; neural networks can learn weights but need large data and parameters.
Context Sensitivity	✗ No	Each word has exactly one fixed vector regardless of context.
Generalization Capability	✗ No	Cannot generalize to unseen words or capture relationships between words.

Word Representation: *WordNet*

Vectors from annotated discrete properties

e.g. “nodule”

$$v_{nodule} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \begin{array}{l} \text{(plural-noun)} \\ \text{(verb)} \\ \text{(synonym-1)} \\ \cdot \\ \cdot \\ \cdot \\ \text{(synonym-n)} \end{array}$$

Word Representation: *WordNet*

WordNet

- A large lexical database of English, developed under the direction of cognitive psychologist George A. Miller at Princeton University. First released in 1985.

Key Features:

- **Synsets (Synonym Sets):** Each synset represents a distinct concept and groups together words that express that concept. For example, {car, auto, automobile, machine, motorcar} is a synset.
- **Semantic Relations:** WordNet organizes synsets into a network of semantic relations.
 - **Hypernyms:** The more general form of a concept (e.g., 'vehicle' is a hypernym of 'car').
 - **Hyponyms:** The more specific form of a concept (e.g., 'car' is a hyponym of 'vehicle').
 - **Meronyms:** Part-whole relationships (e.g., 'wheel' is a meronym of 'car').
 - **Holonyms:** The opposite of meronyms, representing the whole to a part (e.g., 'car' is a holonym of 'wheel').
 - **Antonyms:** Words with opposite meanings.
- **Rich Lexical Information:** WordNet covers various parts of speech such as nouns, verbs, adjectives, and adverbs, each with its own network of semantic relations.
- **Example Sentences:** Many synsets in WordNet are accompanied by example sentences, illustrating the use of a word in a particular sense.

Word Representation: *WordNet*

Vectors from annotated discrete properties

e.g. “nodule”

$$v_{nodule} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \begin{array}{l} \text{(plural-noun)} \\ \text{(verb)} \\ \text{(synonym-1)} \\ \cdot \\ \cdot \\ \cdot \\ \text{(synonym-n)} \end{array}$$

Limitations

- Sparse representation
- Limited Coverage of Vocabulary
- Ignores context (surrounding words)
- Inflexible to deal with new words due to fixed length

Word Representation: *Bag-of-Words (BOW)*

Frequency-based Representation

Bag-of-Words (BOW)

- Construct an $n \times t$ document-term matrix, where n is the number of documents, and t is the number of unique terms.
- Each column represents a unique term, and each cell (i, j) represents how many of term j are in document i .

Word Representation: *Bag-of-Words (BOW)*

Frequency-based Representation

Bag-of-Words (BOW)

1 I like solving interesting problems.
2 What is machine learning?
3 I'm not sure.
4 Machien lerning predicts eveyrthing.

	everyrthing	interesting	learning	lerning	like	Machien	machine	not	predicts	problems	solving	sure	What
1	0	1	0	0	1	0	0	0	0	1	1	0	0
2	0	0	1	0	0	0	1	0	0	0	0	0	1
3	0	0	0	0	0	0	0	1	0	0	0	1	0
4	1	0	0	1	0	1	0	0	1	0	0	0	0

Word Representation: *Bag-of-Words (BOW)*

Frequency-based Representation

Bag-of-Words (BOW)

Limitations

- High-dimensional
- Sparsity
- Biased toward frequent words.

Word Representation: *Bag-of-Words (BOW)*

Frequency-based Representation

Bag-of-Words (BOW)

Characteristic	Satisfied?	Explanation
Semantic Richness	✗ No	Words are still independent dimensions; <i>cat</i> and <i>dog</i> are unrelated unless statistics are added (e.g., TF-IDF, PMI).
Syntactic Awareness	✗ No	Word order and grammar are discarded entirely.
Dimensionality and Sparsity	✗ No	Vectors are high-dimensional and sparse (one dimension per vocabulary term).
Robustness to Noise	✗ No	Typos and variants create new dimensions and fragment statistics.
Learnability	⚠ Partial	Works reasonably well with linear models (e.g., Naive Bayes, Logistic Regression).
Context Sensitivity	✗ No	Words have the same representation regardless of sentence or surrounding words.
Generalization Capability	⚠ Limited	Can generalize across documents statistically, but not across unseen words or meanings.

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

- **Term Frequency:** TF of a term or word is the number of times the term appears in a document compared to the total number of words in the document.

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

- **Inverse Document Frequency:** IDF of a term reflects the proportion of documents in the corpus that contain the term. Words unique to a small percentage of documents (e.g., technical jargon terms) receive higher importance values than words common across all documents (e.g., a, the, and).

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

- $$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$
- $$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

$$TF-IDF = TF * IDF$$

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

Document 1: ["apple", "and", "banana", "are", "fruits"]

Document 2: ["i", "like", "to", "eat", "apple", "and", "banana"]

Document 3: ["banana", "and", "orange", "juice", "are", "delicious"]

Step 1: Preprocess the Documents

Tokenizing: Splitting sentences into words.

Lowercasing: Converting all words to lowercase.

Removing Punctuation: Eliminating periods, commas, etc.

[Optional] **Removing Stop Words:** For simplicity, we'll keep all words.

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

Step 2: Build the Vocabulary

Vocabulary: ["apple", "and", "banana", "are", "fruits", "i", "like", "to", "eat", "orange", "juice", "delicious"]

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

Step 3: Calculate Term Frequency (TF)

Document 1

Term	Frequency	TF (Term Count / Total Terms)
apple	1	$1 / 5 = 0.20$
and	1	$1 / 5 = 0.20$
banana	1	$1 / 5 = 0.20$
are	1	$1 / 5 = 0.20$
fruits	1	$1 / 5 = 0.20$

Document 2

Term	Frequency	TF (Term Count / Total Terms)
i	1	$1 / 7 \approx 0.14$
like	1	$1 / 7 \approx 0.14$
to	1	$1 / 7 \approx 0.14$
eat	1	$1 / 7 \approx 0.14$
apple	1	$1 / 7 \approx 0.14$
and	1	$1 / 7 \approx 0.14$
banana	1	$1 / 7 \approx 0.14$

Document 3

Term	Frequency	TF (Term Count / Total Terms)
banana	1	$1 / 6 \approx 0.17$
and	1	$1 / 6 \approx 0.17$
orange	1	$1 / 6 \approx 0.17$
juice	1	$1 / 6 \approx 0.17$
are	1	$1 / 6 \approx 0.17$
delicious	1	$1 / 6 \approx 0.17$

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

Step 4: Calculate Inverse Document Frequency (IDF)

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

Term	Document Frequency (DF)	IDF
apple	2	$\log(3/2) \approx 0.176$
and	3	$\log(3/3) = 0$
banana	3	$\log(3/3) = 0$
are	2	$\log(3/2) \approx 0.176$
fruits	1	$\log(3/1) \approx 1.0986$
i	1	$\log(3/1) \approx 1.0986$
like	1	$\log(3/1) \approx 1.0986$
to	1	$\log(3/1) \approx 1.0986$
eat	1	$\log(3/1) \approx 1.0986$
orange	1	$\log(3/1) \approx 1.0986$
juice	1	$\log(3/1) \approx 1.0986$
delicious	1	$\log(3/1) \approx 1.0986$

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

$$TF-IDF = TF * IDF$$

Step 5: Calculate TF-IDF

Document 1

Term	TF	IDF	TF-IDF (TF × IDF)
apple	0.20	0.176	$0.20 \times 0.176 \approx 0.0352$
and	0.20	0	$0.20 \times 0 = 0$
banana	0.20	0	$0.20 \times 0 = 0$
are	0.20	0.176	$0.20 \times 0.176 \approx 0.0352$
fruits	0.20	1.0986	$0.20 \times 1.0986 \approx 0.2197$

Document 2

Term	TF	IDF	TF-IDF (TF × IDF)
i	0.14	1.0986	$0.14 \times 1.0986 \approx 0.1538$
like	0.14	1.0986	$0.14 \times 1.0986 \approx 0.1538$
to	0.14	1.0986	$0.14 \times 1.0986 \approx 0.1538$
eat	0.14	1.0986	$0.14 \times 1.0986 \approx 0.1538$
apple	0.14	0.176	$0.14 \times 0.176 \approx 0.0246$
and	0.14	0	$0.14 \times 0 = 0$
banana	0.14	0	$0.14 \times 0 = 0$

Document 3

Term	TF	IDF	TF-IDF (TF × IDF)
banana	0.17	0	$0.17 \times 0 = 0$
and	0.17	0	$0.17 \times 0 = 0$
orange	0.17	1.0986	$0.17 \times 1.0986 \approx 0.1862$
juice	0.17	1.0986	$0.17 \times 1.0986 \approx 0.1862$
are	0.17	0.176	$0.17 \times 0.176 \approx 0.0299$
delicious	0.17	1.0986	$0.17 \times 1.0986 \approx 0.1862$

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Example

Document 1: "Apple and banana are fruits."

Document 2: "I like to eat apple and banana."

Document 3: "Banana and orange juice are delicious."

Step 6: Create TF-IDF Vectors

Each document can now be represented as a vector of TF-IDF values corresponding to each term in the vocabulary.

Vocab = ["apple", "and", "banana", "are", "fruits", "i", "like", "to", "eat", "orange", "juice", "delicious"]

Document 1: [0.0352, 0, 0, 0.0352, 0.2197, 0, 0, 0, 0, 0, 0, 0]

Document 2: [0.0246, 0, 0, 0, 0, 0.1538, 0.1538, 0.1538, 0.1538, 0, 0, 0]

Document 3: [0, 0, 0, 0.0299, 0, 0, 0, 0, 0, 0.1862, 0.1862, 0.1862]

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

$$TF-IDF = TF * IDF$$

Limitations

- Lack of context

Word Representation: *TF-IDF*

Frequency-based Representation

Term-Frequency - Inverse Document Frequency (TF-IDF)

Characteristic	Satisfied?	Explanation
Semantic Richness	⚠ Limited	Captures <i>importance</i> of words, not meaning; <i>cat</i> and <i>dog</i> are still independent dimensions.
Syntactic Awareness	✗ No	Word order and grammar are ignored.
Dimensionality and Sparsity	✗ No	Still very high-dimensional and sparse (vocab-sized vectors).
Robustness to Noise	✗ No	Typos and spelling variants create separate features and dilute statistics.
Learnability	✅ Yes	Works very well with linear models; strong baseline for classification and retrieval.
Context Sensitivity	✗ No	A word has the same dimension regardless of sentence-level context or sense.
Generalization Capability	⚠ Limited	Generalizes across documents via weighting, but not to unseen words or semantic relations.

Word Representation: *Co-Occurrence Vector*

Frequency-based Representation

Co-Occurrence Vector

- word-word co-occurrence matrix : number of times two words occur together in a context window
- Term-document matrix (BoW): number of times a word occurs in a document

Word Representation: *Co-Occurrence Vector*

Frequency-based Representation

Word-Word Co-occurrence Matrix

sugar, a sliced lemon, a tablespoonful of
their enjoyment. Cautiously she sampled her first
well suited to programming on the digital
for the purpose of gathering data and

apricot
pineapple
computer.
information

preserve or jam, a pinch each of,
and another fruit whose taste she likened
In finding the optimal R-stage policy from
necessary for the study authorized in the

context

word

	aardvark	computer	data	pinch	result	sugar	...
apricot	0	0	0	1	0	1	
pineapple	0	0	0	1	0	1	
digital	0	2	1	0	1	0	
information	0	1	6	0	4	0	
...	...						

Word Representation: *Co-Occurrence Vector*

Frequency-based Representation

Co-Occurrence Vector

Limitations

- Raw frequency is very skewed and not very discriminative
- Suffers from sparsity

Word Representation: *Pointwise Mutual Information*

Frequency-based Representation

Pointwise Mutual Information (PMI)

A statistical approach to used in *Information Theory* to quantify the association between two events or variables (e.g. two words).

Measure of how often two pointwise mutual information events x and y occur, compared with what we would expect if they were independent

$$\text{PMI}(x, y) = \log \left(\frac{P(x, y)}{P(x)P(y)} \right)$$

- $P(x, y)$ is the probability of both words x and y occurring together (co-occurrence) in a given context.
- $P(x)$ and $P(y)$ are the individual probabilities of words x and y occurring in the corpus, respectively.

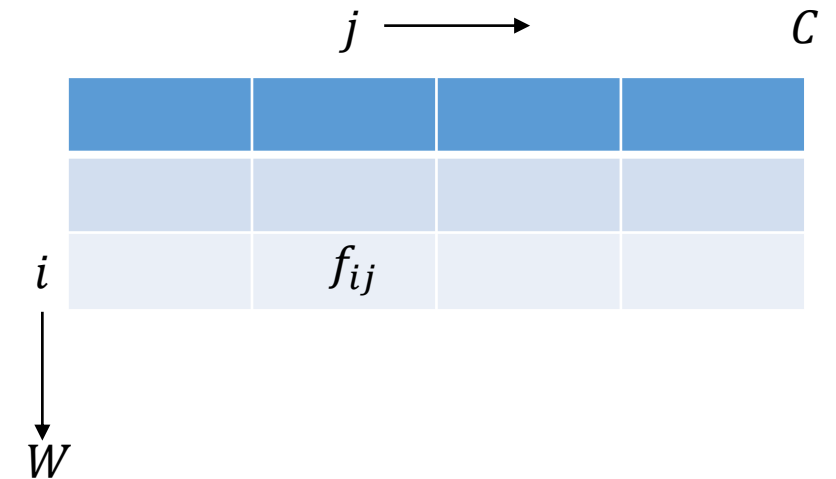
Word Representation: *Pointwise Mutual Information*

Frequency-based Representation

Pointwise Mutual Information (PMI)

$$PMI(w, c) = \log_2 \frac{P(w, c)}{P(w) \cdot P(c)}$$

$$P(i, j) = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$
$$P(i) = \frac{\sum_{j=1}^C f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$
$$P(j) = \frac{\sum_{i=1}^W f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$



Word Representation: *Pointwise Mutual Information*

Frequency-based Representation

Pointwise Mutual Information (PMI)

	aardvark	computer	data	pinch	result	sugar	...
apricot	0	0	0	1	0	1	
pineapple	0	0	0	1	0	1	
digital	0	2	1	0	1	0	
information	0	1	6	0	4	0	
...	...						

$$P(w, c)$$

	computer	data	pinch	result	sugar
apricot	0.0	0.0	0.05	0.0	0.05
pineapple	0.0	0.0	0.05	0.0	0.05
digital	0.11	0.05	0.0	0.05	0.0
information	0.05	0.32	0.0	0.21	0.0
...					

$$P(w)$$

0.11

0.11

0.21

0.58

...

$$P(c)$$

0.16

0.37

0.11

0.26

0.11

Word Representation: *Pointwise Mutual Information*

Frequency-based Representation

Pointwise Mutual Information (PMI)

$$PMI(w, c)$$

	computer	data	pinch	result	sugar
apricot					
pineapple					
digital					
information					
...					

Word Representation: *Pointwise Mutual Information*

Frequency-based Representation

Pointwise Mutual Information (PMI)

Limitations

- Biased towards infrequent events

Word Representation: *Context matters*



“You shall know a word by the company it keeps”

J.R.Firth 1957

Word Representation: *Context matters*

1. **Polysemy** (Same word, multiple related meanings)

Example: The term "cold" in a clinical setting.

It could refer to a common viral infection (common cold).

It might also describe the temperature sensation (the patient feels cold).

2. Homonymy

3. Metonymy

4. Synonymy

Word Representation: *Context matters*

1. Polysemy

2. **Homonymy** (Same word, multiple unrelated meanings)

Example: The term "lead" in clinical notes.

It could refer to the metal "lead" in a context discussing heavy metal poisoning.

It might also be used as a verb, such as "lead" in "lead to complications," indicating causation.

3. Metonymy

4. Synonymy

Word Representation: *Context matters*

1. Polysemy
2. Homonymy
3. **Metonymy** (A word used as a substitute for something else with which it's closely related)

Example: The use of "bottle" in a pediatric context.

"Bottle" might be used metonymically to refer to bottle feeding, encompassing the entire activity of feeding a baby with a bottle, including the type of milk or formula used.

1. Synonymy

Word Representation: *Context matters*

1. Polysemy
2. Homonymy
3. Metonymy
4. **Synonymy** (Different words with similar meanings)

Example: Terms like "myocardial infarction," "heart attack," and "cardiac arrest."

While these terms are often used interchangeably in casual conversation, they have distinct meanings in a clinical context.

"Myocardial infarction" refers to the blockage of blood flow to a part of the heart, "heart attack" is a common term for this condition, and "cardiac arrest" refers to the sudden loss of heart function.

Word Representation: *so far what we learned*

IDEAL Word Representation

One-hot vector → dense representation

TF-IDF → take context into account

Co-Occurrence → distribution-driven (rather than freq)

PMI → continuous vector

Word Representation: *Distributed Representation*

Distributional Hypothesis

Zellig S. Harris (1954): An American linguist who formalized the hypothesis, stating that words with similar distributional properties (i.e., occurring in similar contexts) are likely to have similar meanings.

J.R. Firth (1957): A British linguist who encapsulated the idea with the phrase, *"You shall know a word by the company it keeps."*

- Distributional Hypothesis led to distributional semantic models that represent words as numeric vectors based on pattern analysis of massive text corpora.

Word Representation: *Distributed Representation*

Distributional Hypothesis

Zellig S. Harris (1954): An American linguist who formalized the hypothesis, stating that words with similar distributional properties (i.e., occurring in similar contexts) are likely to have similar meanings.

J.R. Firth (1957): A British linguist who encapsulated the idea with the phrase, *"You shall know a word by the company it keeps."*

- Distributional Hypothesis led to distributional semantic models that represent words as numeric vectors based on pattern analysis of massive text corpora.

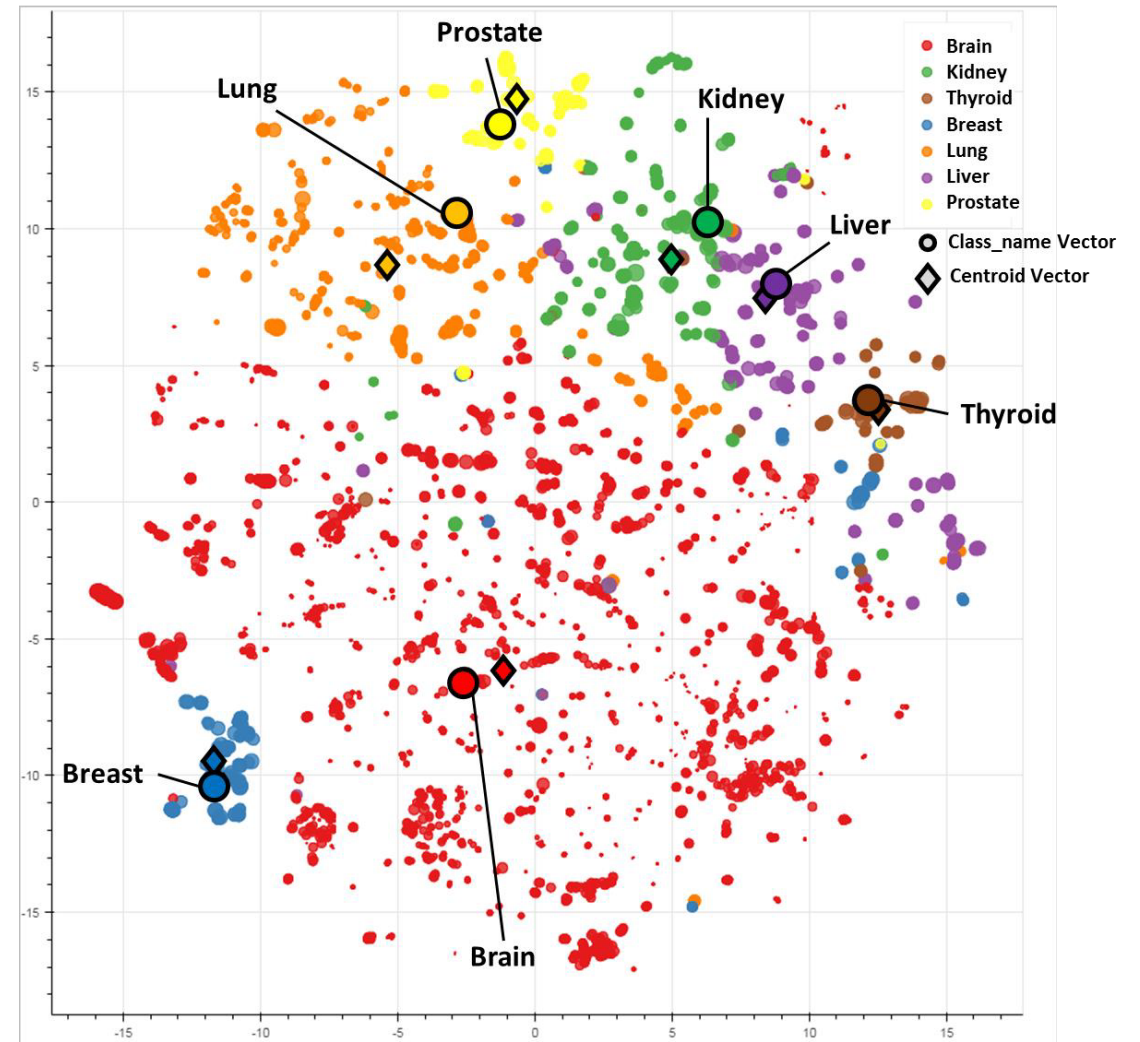
Machine Learning

Word Representation: *Distributed Representation*

Word Embedding

Definition: representation of words as dense, continuous vectors in a multi-dimensional space.

Mathematically, an embedding space, or latent space, is defined as a manifold in which similar items are positioned closer to one another than less similar items. In this case, words that are semantically similar should have similar embedded vectors and thus be closer together in the space.



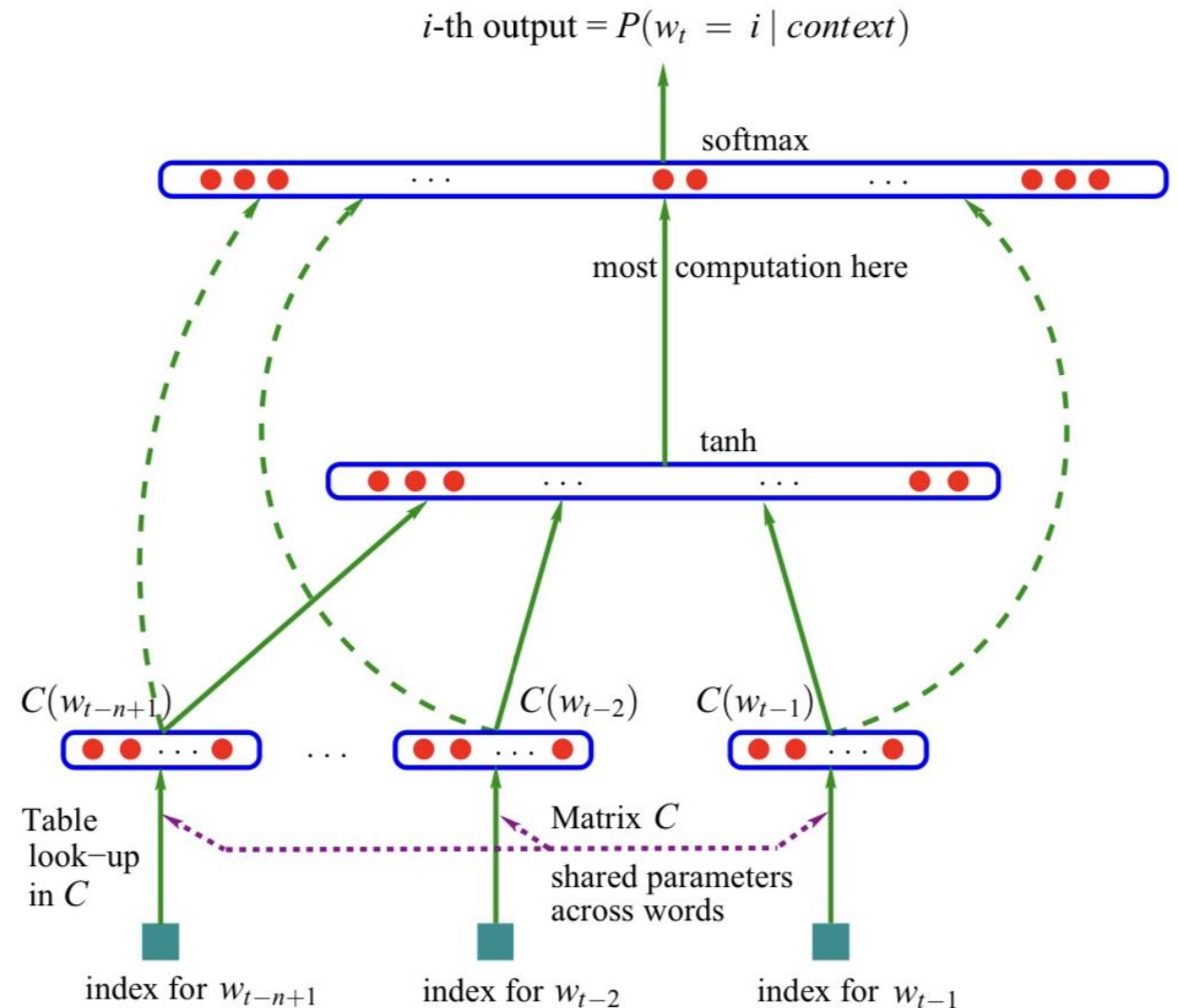
Word Representation: *Distributed Representation*

Word Representation as Vector

[Bengio et al. 2003](#):

“A Neural Probabilistic Language Model”

tackle what's known as the *curse of dimensionality*



Word Representation: *Distributed Representation*

Word to Vector Representation (Word2Vec)

- Mikolov *et al.*, 2013 <https://arxiv.org/pdf/1310.4546.pdf>
- Trained to predict the probabilities of a word being a context word for the given target.



Tomáš Mikolov

Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context

While I am teaching this lecture, you are

Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context

While I am teaching this lecture, you are

context target

Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context

While I am teaching this lecture,

you are

context

.....

target

strong
welcome
sleeping
eating
listening
...

Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context

While I am teaching this lecture, you are

.....

context

target

Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context

While I am teaching this lecture, you are

context

.....

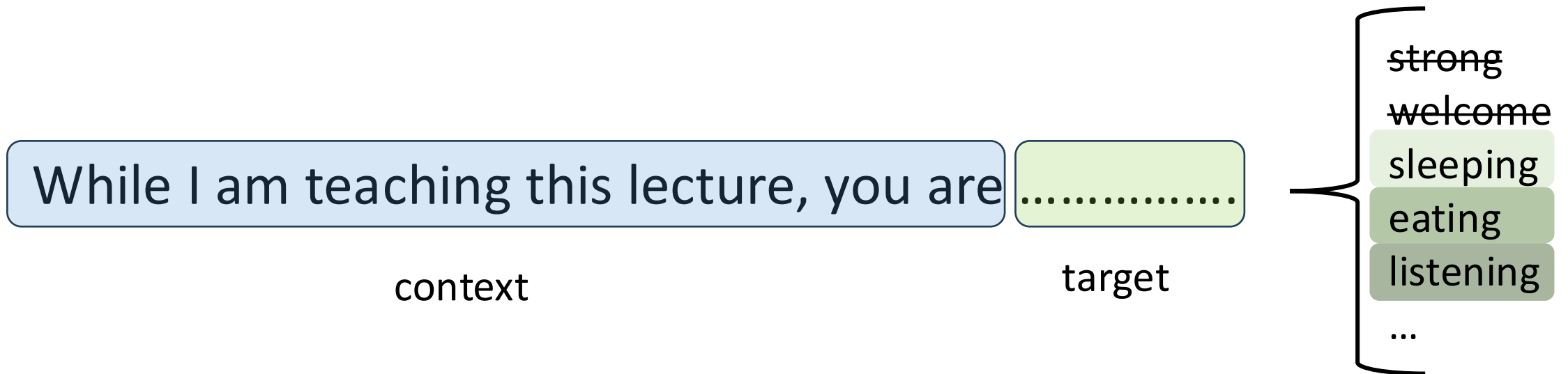
target

strong
welcome
sleeping
eating
listening
...

Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context



Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context

While I am teaching this lecture, you arenotes.

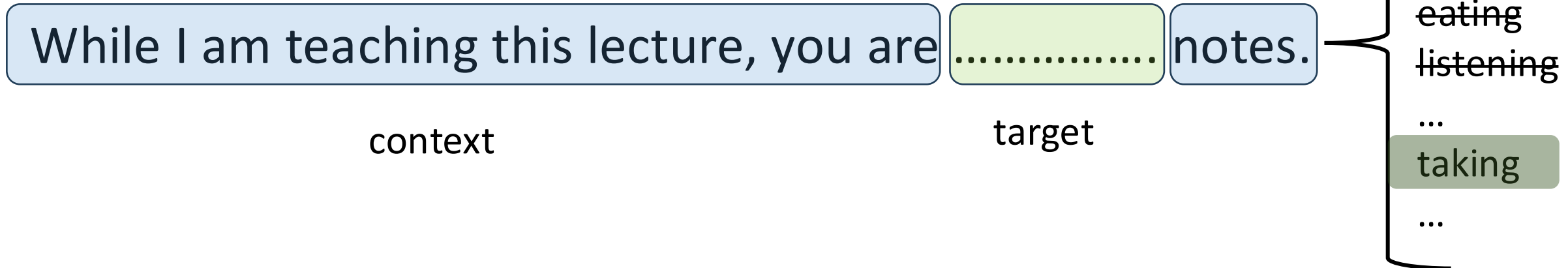
context

target

Word Representation: *word2vec*

Language Model

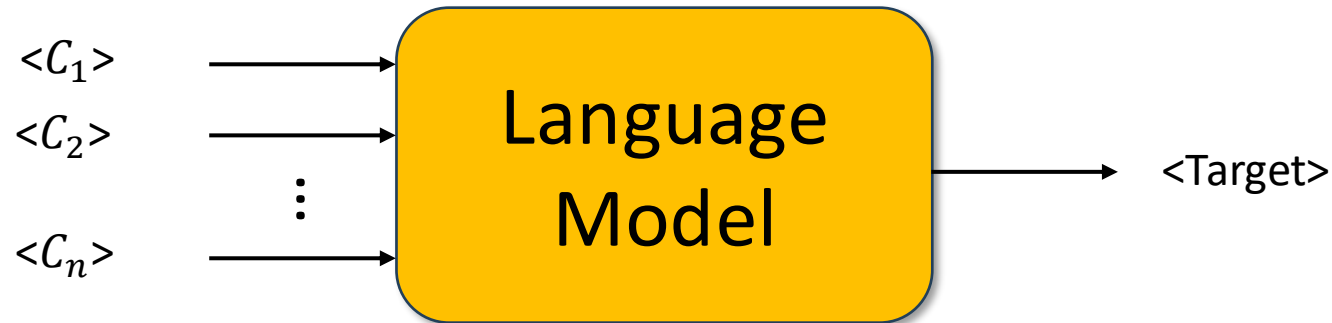
- Given a context (a few words), predict the next word following context



Word Representation: *word2vec*

Language Model

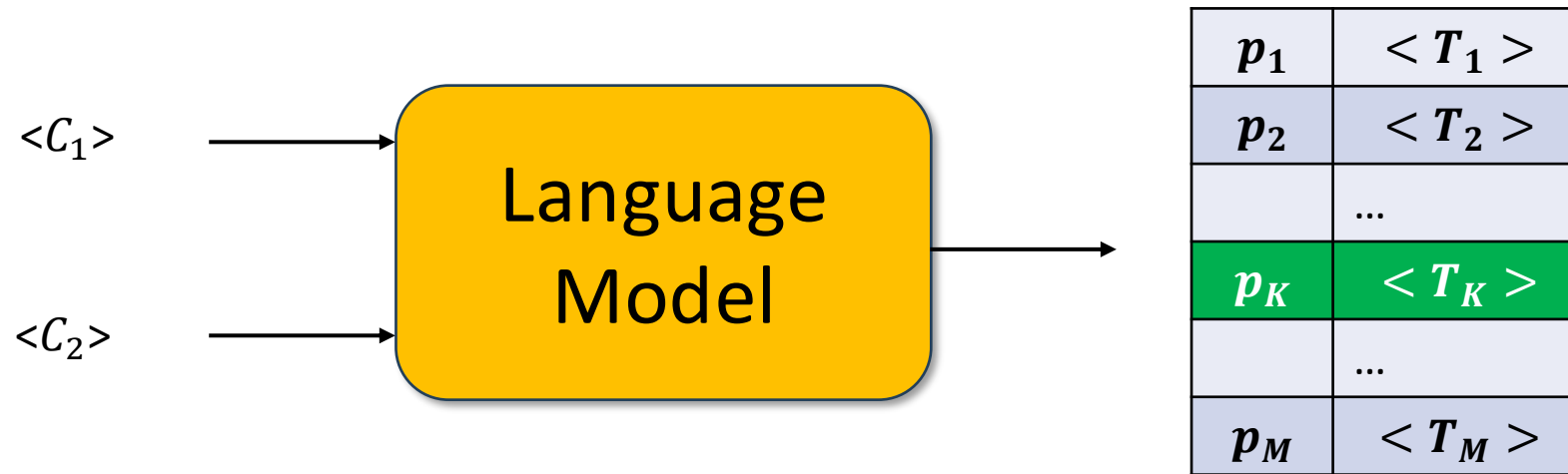
- Given a context (a few words), predict the next word following context



Word Representation: *word2vec*

Language Model

- Given a context (a few words), predict the next word following context



$$\sum_{i=1}^M p_i = 1$$

Word Representation: *word2vec*

Language Model

Training

- **Prepare training data**

Word Representation: *word2vec*

Language Model

Training

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

- Prepare training data



Training
corpus

Word Representation: *word2vec*

Language Model

Training

wikipedia	is	a	free	content	online	encyclopedia
$\langle C_1 \rangle$	$\langle C_2 \rangle$	$\langle T_1 \rangle$				

- Prepare training data



Training
corpus

Word Representation: *word2vec*

Language Model

Training

- Prepare training data



Training
corpus

wikipedia	is	a	free	content	online	encyclopedia
wikipedia	is	a	free	content	online	encyclopedia
wikipedia	is	a	free	content	online	encyclopedia
wikipedia	is	a	free	content	online	encyclopedia
wikipedia	is	a	free	content	online	encyclopedia

Word Representation: *word2vec*

Language Model

Training

- Prepare training data



Training
corpus

wikipedia	is	a	free	content	online	encyclopedia
wikipedia	is	a	free	content	online	encyclopedia
wikipedia	is	a	free	content	online	encyclopedia



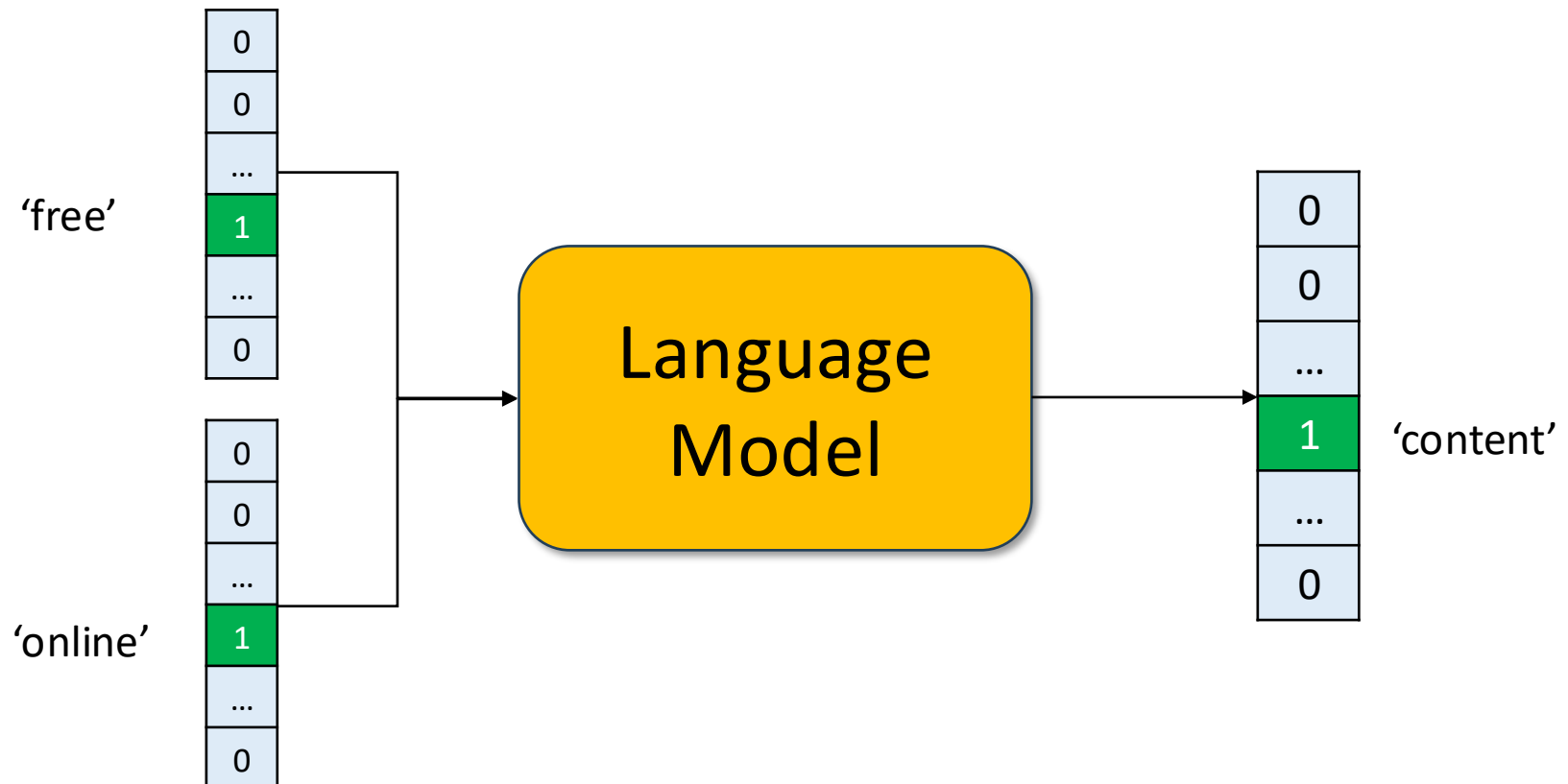
Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Process



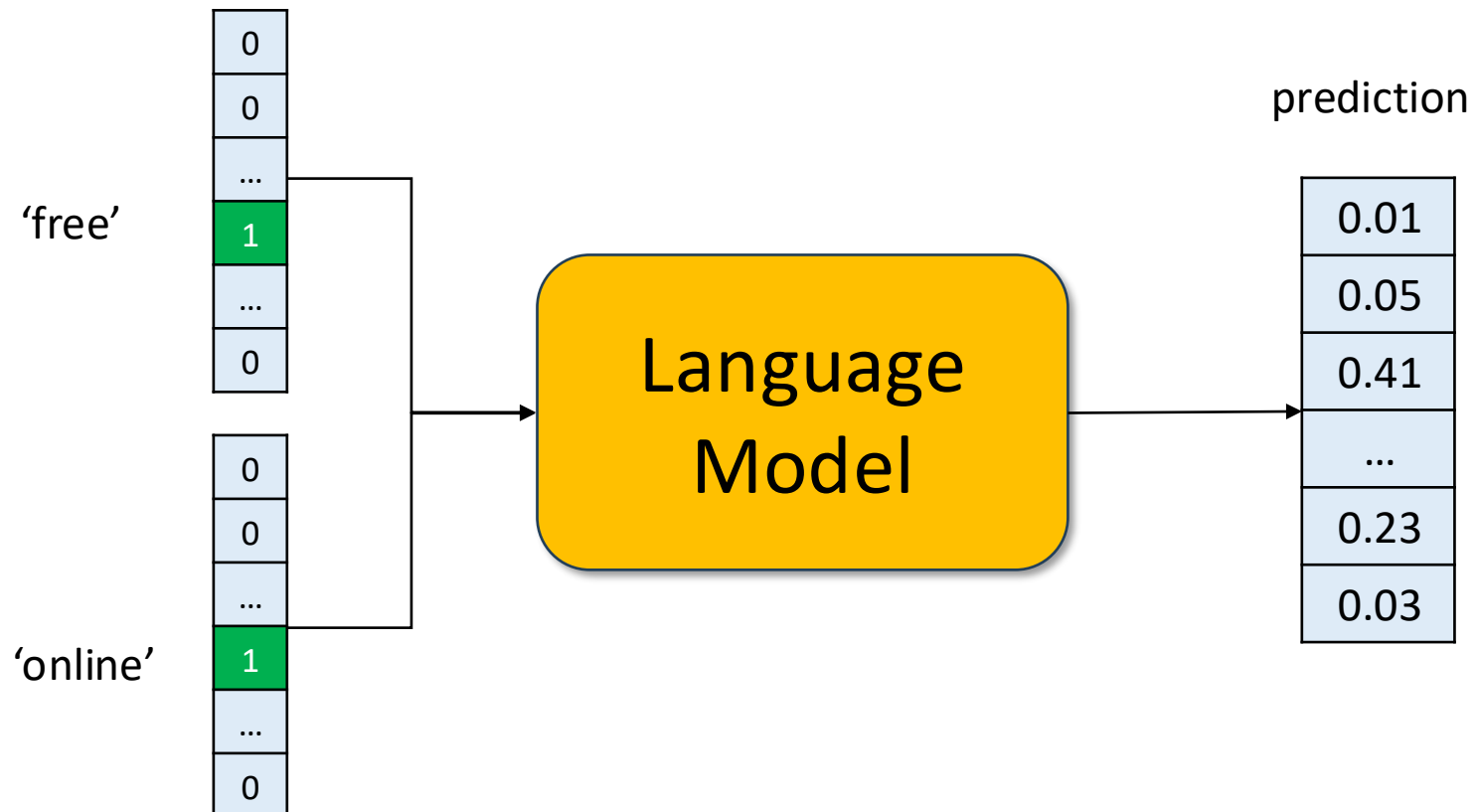
Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Process



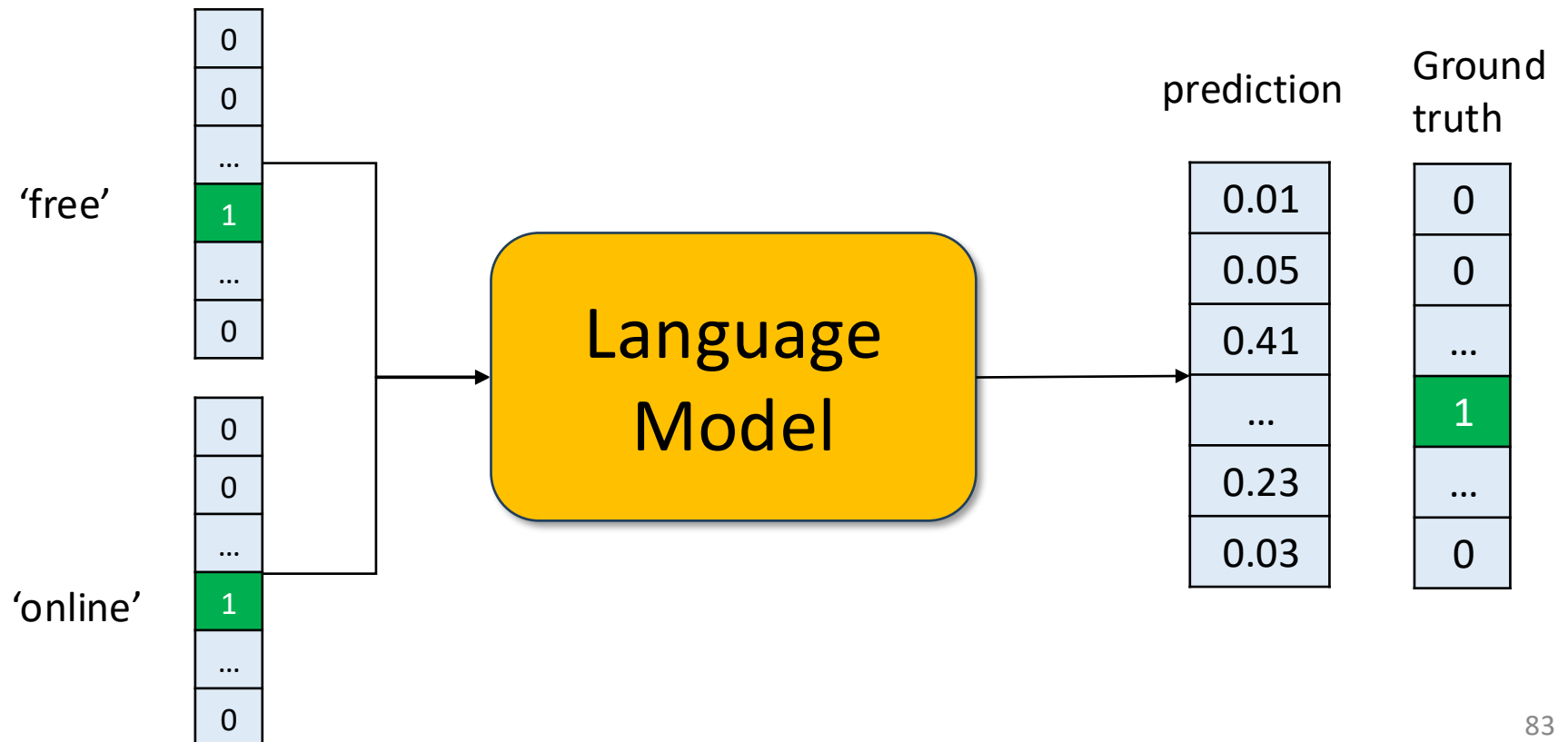
Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Process



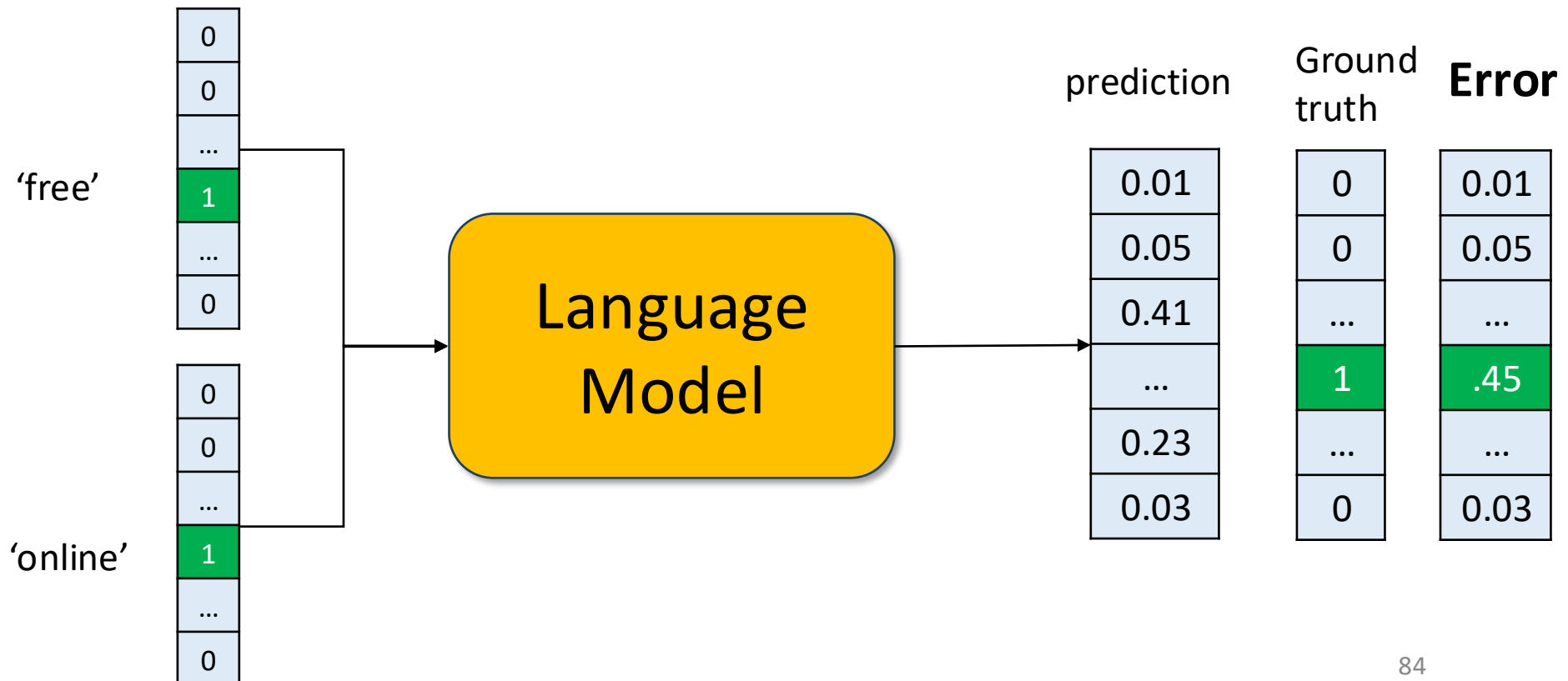
Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Process



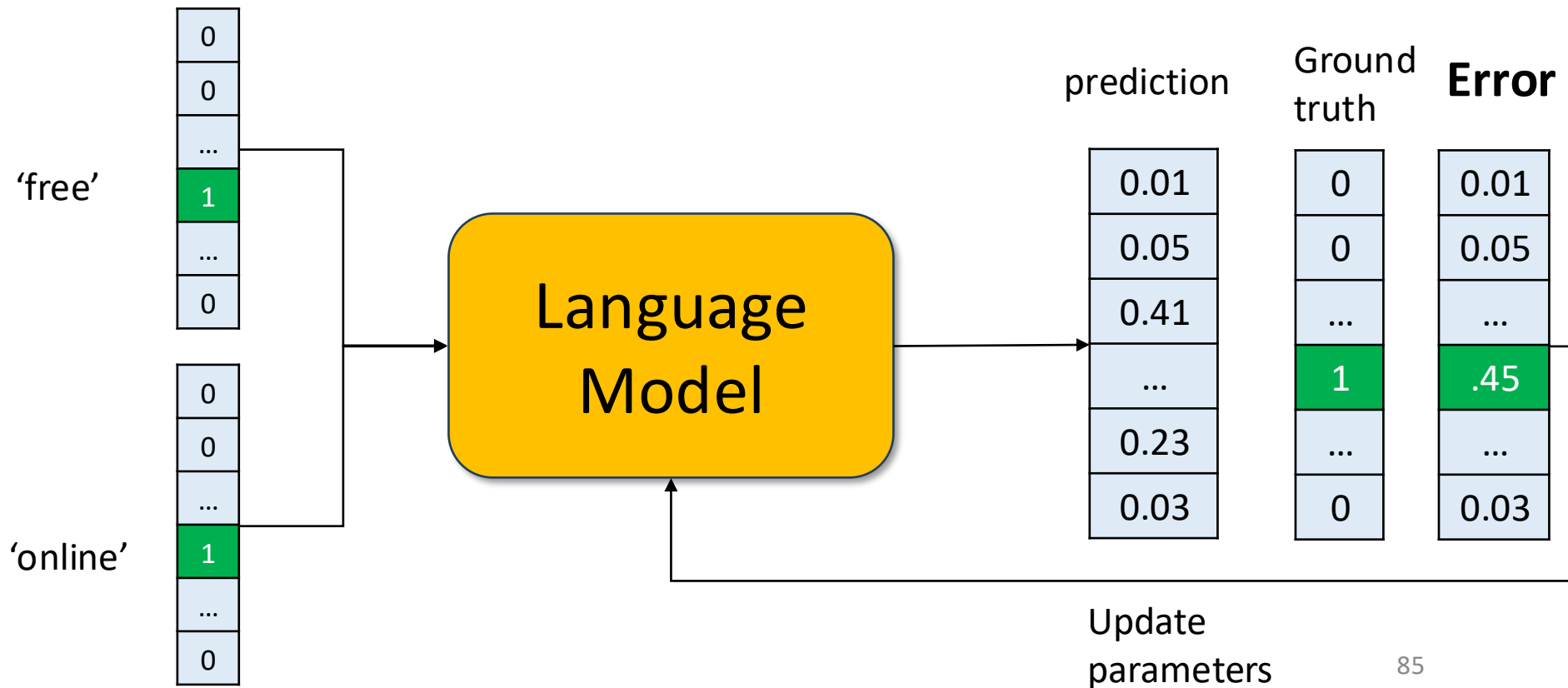
Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Process



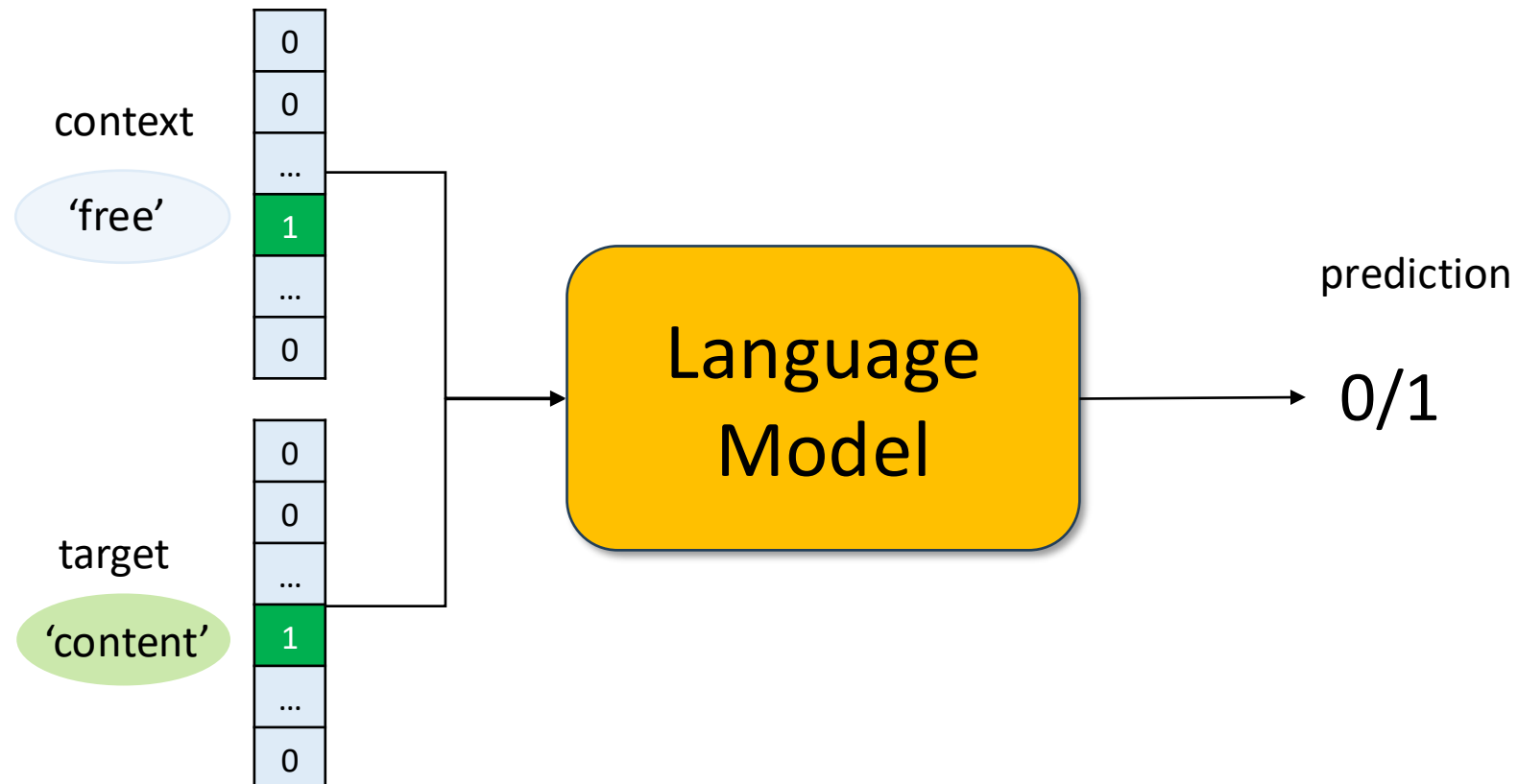
Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Given input X, Y, predict if they are neighbors



Word Representation: *word2vec*

Language Model

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Training

- Negative Sampling

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

wikipedia	is	a	free	content	online	encyclopedia
-----------	----	---	------	---------	--------	--------------

Word Representation: *Distributed Representation*

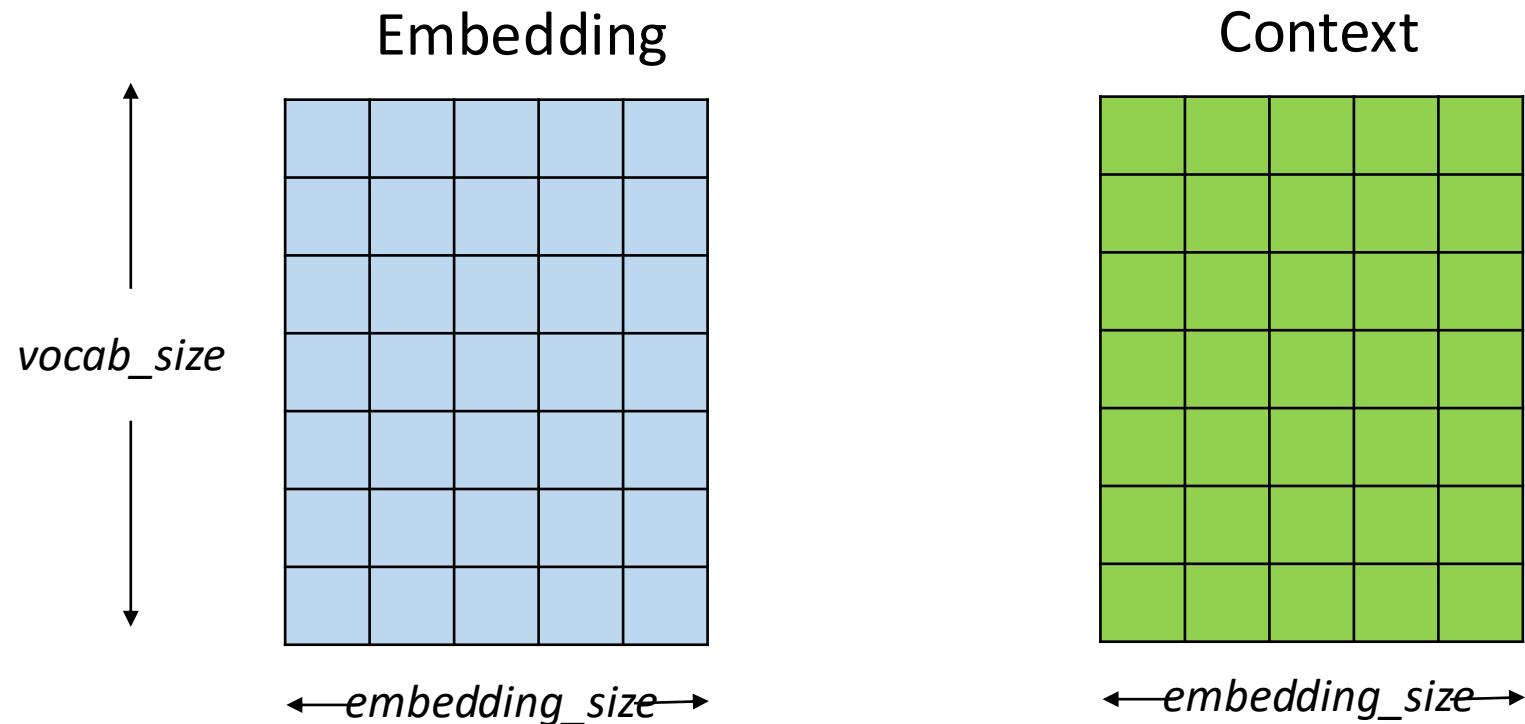
Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (*vocab_size*)

Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (`vocab_size`)
- 2) Create two matrices

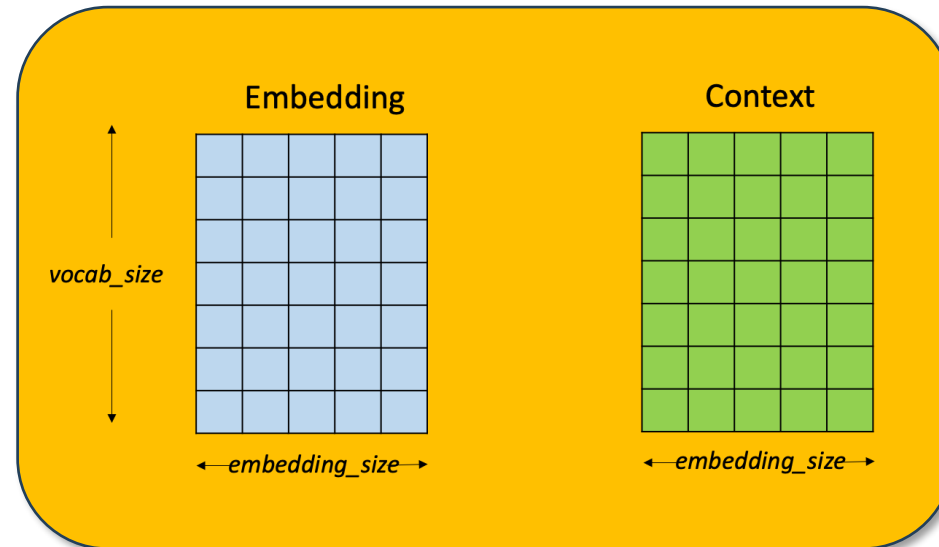


Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (`vocab_size`)
- 2) Create two matrices
- 3) In each training step, we take one positive example and its associated negative examples

input	output	target
Word_1	Word_2	1
		0
		0
		0
		...

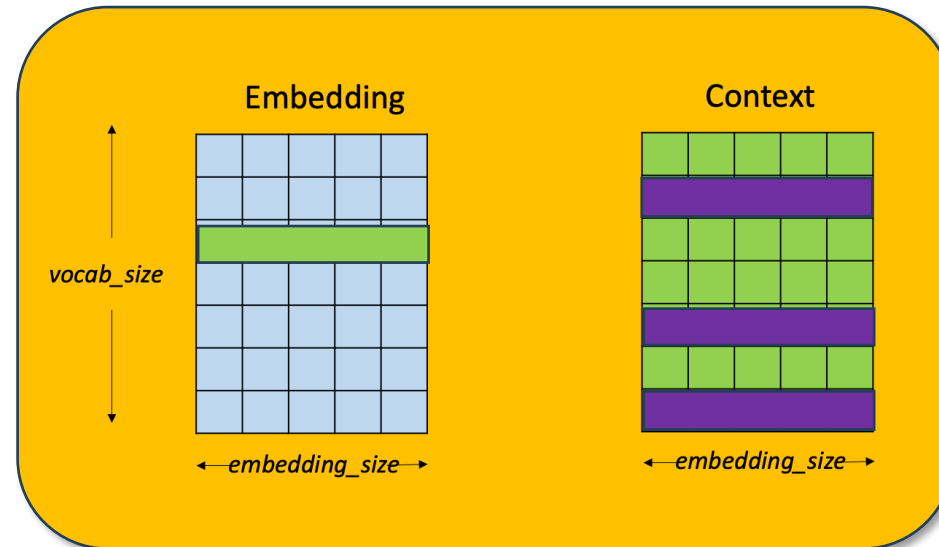


Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (`vocab_size`)
- 2) Create two matrices
- 3) In each training step, we take one positive example and its associated negative examples
- 4) look up their embeddings

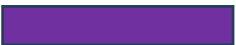
input	output	target
Word_1	Word_2	1
		0
		0
		0
		...



Word Representation: *Distributed Representation*

Word2Vec: Training Process

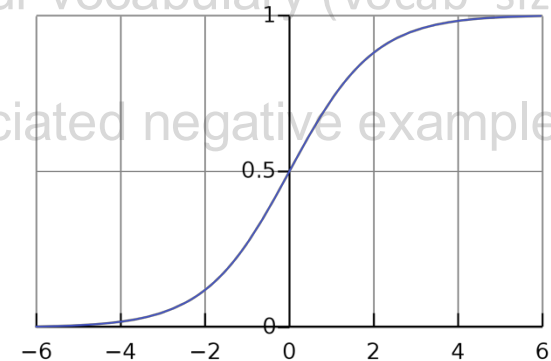
- 1) pre-process the text from training data to determine the size of our vocabulary (vocab_size)
- 2) Create two matrices
- 3) In each training step, we take one positive example and its associated negative examples
- 4) look up their embeddings
- 5) Take the dot product

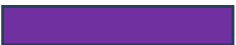
input		output	Target	Dot-product
	•		1	0.2
	•		0	-1.11
	•		0	0.74

Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (vocab_size)
- 2) Create two matrices
- 3) In each training step, we take one positive example and its associated negative examples
- 4) look up their embeddings
- 5) Take the dot product
- 6) Sigmoid: map to positive values between zero and one

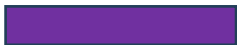


input		output	Target	Dot-product	Sigmoid
	•		1	0.2	0.55
	•		0	-1.11	0.25
	•		0	0.74	0.68

Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (vocab_size)
- 2) Create two matrices
- 3) In each training step, we take one positive example and its associated negative examples
- 4) look up their embeddings
- 5) Take the dot product
- 6) Sigmoid: map to positive values between zero and one
- 7) calculate error

input		output	Target	Dot-product	Sigmoid	Error
	•		1	0.2	0.55	0.45
	•		0	-1.11	0.25	-0.25
	•		0	0.74	0.68	-0.68

Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 1) pre-process the text from training data to determine the size of our vocabulary (vocab_size)
- 2) Create two matrices
- 3) In each training step, we take one positive example and its associated negative examples
- 4) look up their embeddings
- 5) Take the dot product
- 6) Sigmoid: map to positive values between zero and one
- 7) calculate error
- 8) use error score to adjust the embeddings

Word Representation: *Distributed Representation*

Word2Vec: Training Process

- 
- 1) pre-process the text from training data to determine the size of our vocabulary (vocab_size)
 - 2) Create two matrices
 - 3) In each training step, we take one positive example and its associated negative examples
 - 4) look up their embeddings
 - 5) Take the dot product
 - 6) Sigmoid: map to positive values between zero and one
 - 7) calculate error
 - 8) use error score to adjust the embeddings

Word Representation: *Distributed Representation*

Word to Vector Representation (Word2Vec)

- Mikolov *et al.*, 2013 <https://arxiv.org/pdf/1310.4546.pdf>

Word of Wisdom

*“Word2vec doesn’t learn “meaning” directly. It learns **distributional similarity**”*

Words are similar if they occur in similar contexts.

Word Representation: *Distributed Representation*

Word to Vector Representation (Word2Vec)

- Mikolov *et al.*, 2013 <https://arxiv.org/pdf/1310.4546.pdf>

Word of Wisdom

*“Word2vec doesn’t learn “meaning” directly. It learns **distributional similarity**”*

Words are similar if they occur in similar contexts.

what counts as “context”?

controlled almost entirely by two *hyperparameters*.

Word Representation: *Distributed Representation*

Word to Vector Representation (Word2Vec)

- Mikolov *et al.*, 2013 <https://arxiv.org/pdf/1310.4546.pdf>

Word of Wisdom

*“Word2vec doesn’t learn “meaning” directly. It learns **distributional similarity**”*

Words are similar if they occur in similar contexts.

what counts as “context”?

controlled almost entirely by two *hyperparameters*.

Word Representation: *Distributed Representation*

Hyper-parameter vs. Parameter

Parameters:

- **Internal** to the model
- Learned automatically from the data during the model training process.
- Define the model's specific representation learned from the data and are essential for making predictions.

Word Representation: *Distributed Representation*

Hyper-parameter vs. Parameter

Hyper-parameters:

- The configuration variables **external** to the model
- Cannot be directly learned from the data.
- Set prior to the commencement of the learning process and used to control the learning process.
- set by the practitioner and may require tuning to find the optimal model configuration.

Word Representation: *Distributed Representation*

Word2Vec: Hyper-parameters

- **Window Size:** determines how many words on each side of a target word are considered its context.
 - e.g.
 - Window = 2 → very local context
 - Window = 10 → broader topical context
- Number of Negative Samples

Word Representation: *Distributed Representation*

Word2Vec: Hyper-parameters

- **Window Size:** determines how many words on each side of a target word are considered its context.
 - e.g.
 - Window = 2 → very local context
 - Window = 10 → broader topical context
- Number of Negative Samples

Window size

Small (2–3)

Medium (5)

Large (8–15)

Embedding bias

Syntactic

Mixed

Semantic / topical

What it captures

POS, morphology, function

Syntax + semantics

Theme, domain, topic

Word Representation: *Distributed Representation*

Word2Vec: Hyper-parameters

- Window Size
- **Number of Negative Samples** (k): how many “wrong” contexts are shown per positive example.
 - e.g.
 - $k = 2-5 \rightarrow$ small corpora
 - $k = 5-20 \rightarrow$ large corpora (Google News famously used $\sim 5-15$)

Negative sampling trains the model to:

- **Pull together** real (word, context) pairs
- **Push apart** randomly sampled fake pairs

Word Representation: *Distributed Representation*

Word2Vec: Hyper-parameters

- Window Size
- **Number of Negative Samples**

Negative samples

Too few

Moderate

Too many

Effect

Noisy embeddings, weak discrimination

Best balance of speed & quality

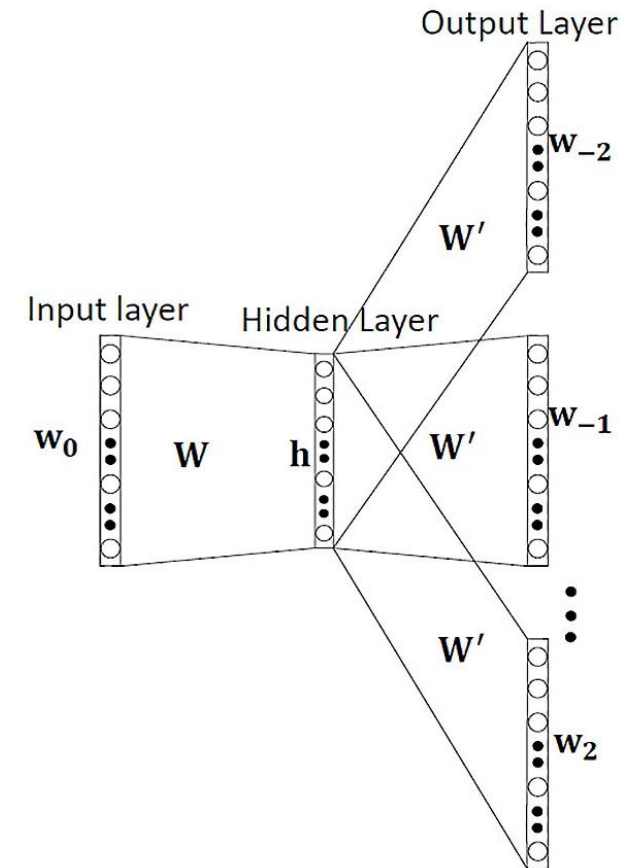
Slower training, marginal gains

Word Representation: *Distributed Representation*

Word2Vec: Skip-gram

Given word, predict context

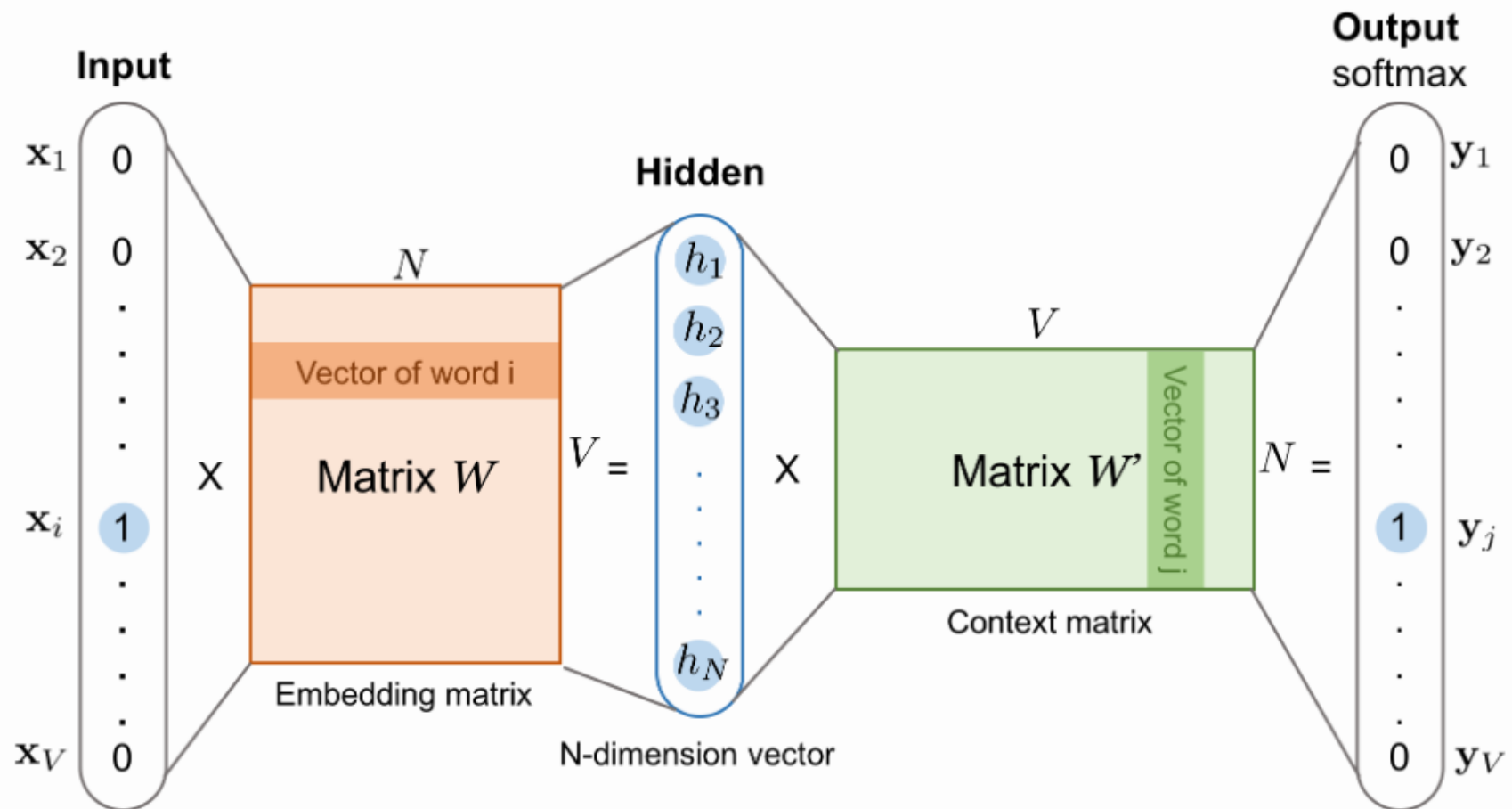
wikipedia	is	a	<?>	content	<?>	encyclopedia
-----------	----	---	-----	---------	-----	--------------



(b) Skip-Gram

Word Representation: *Distributed Representation*

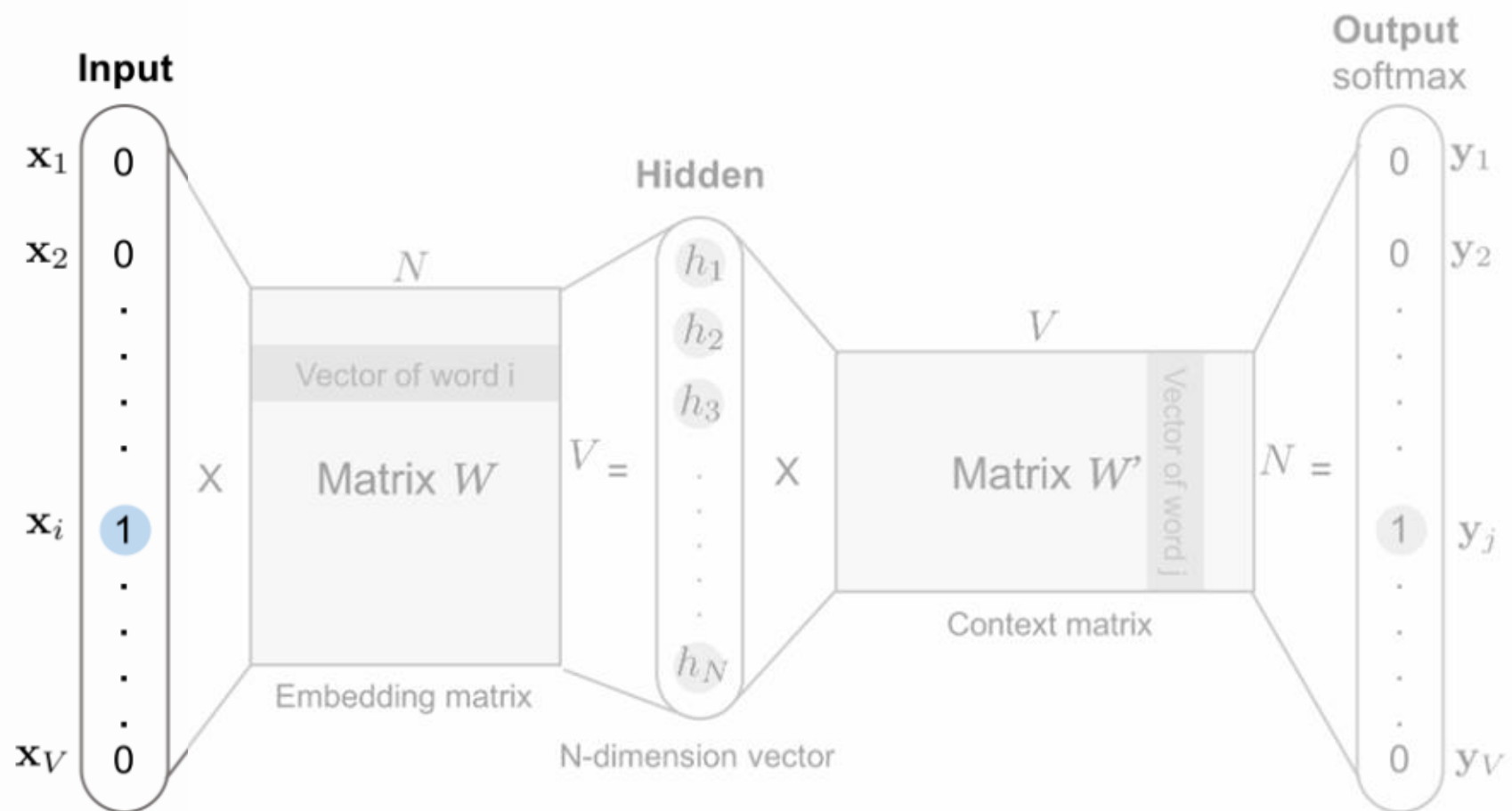
Word2Vec: Indirectly learning representation (through language model task)



Word Representation: *Distributed Representation*

Word2Vec: Indirectly learning representation (through language model task)

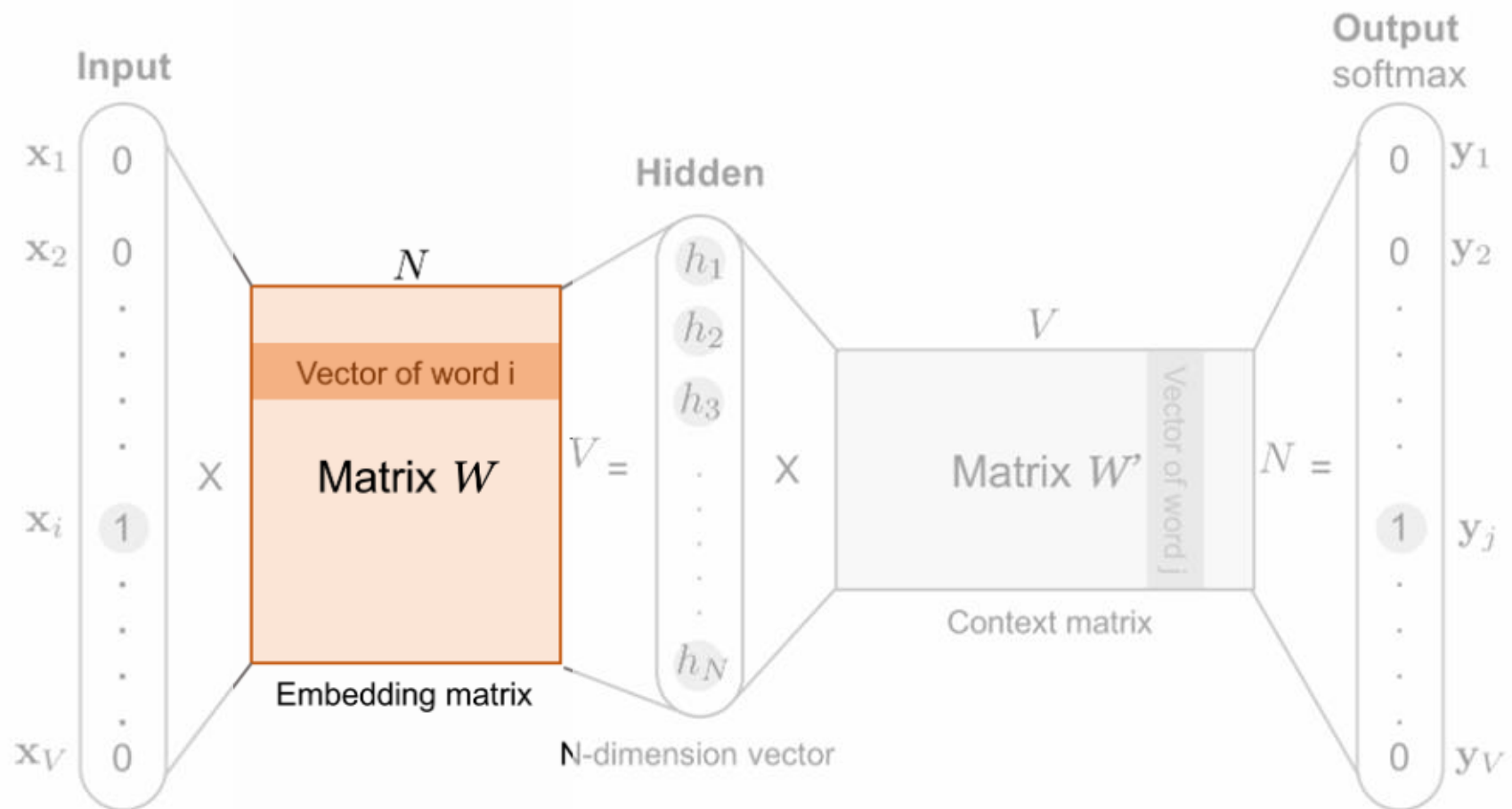
Vocab size: V



Word Representation: *Distributed Representation*

Word2Vec: Indirectly learning representation (through language model task)

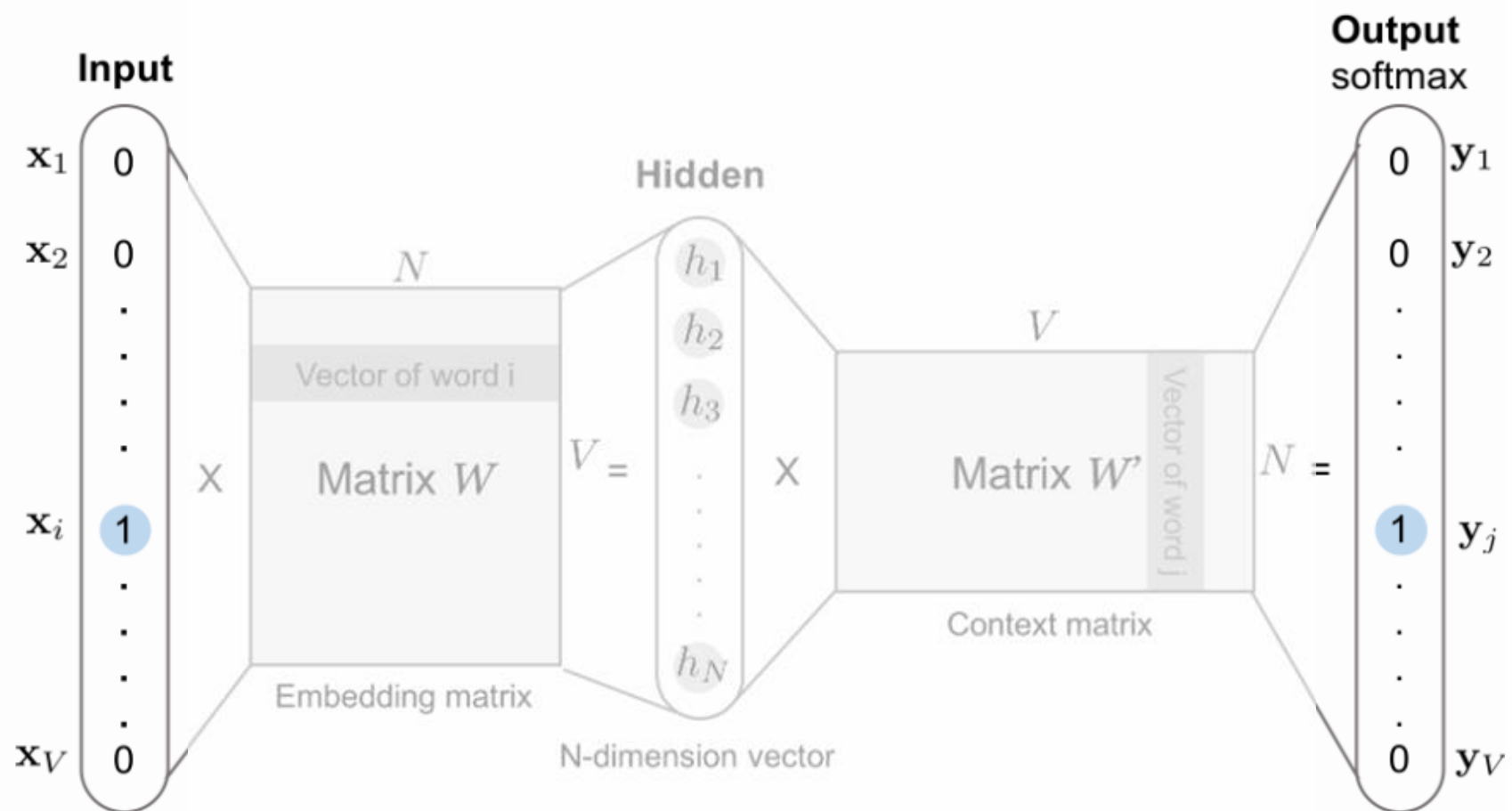
**embedding
size: N**



Word Representation: *Distributed Representation*

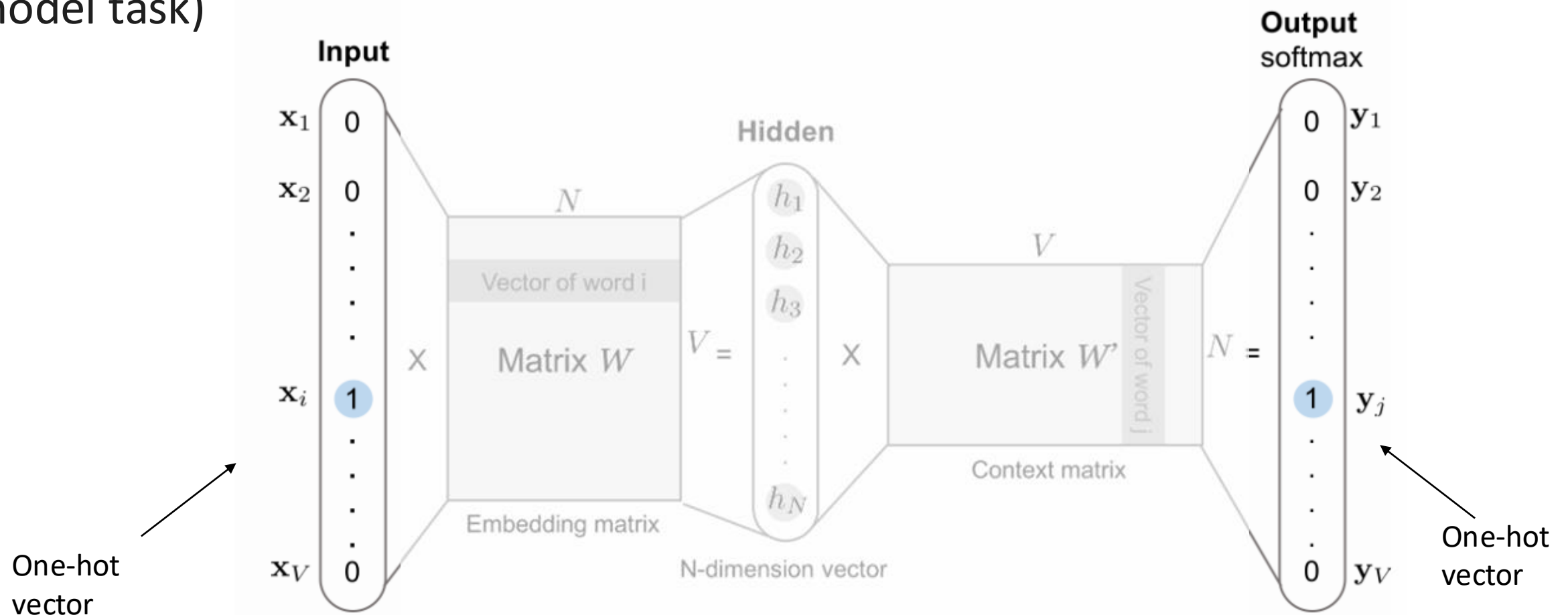
Word2Vec: Indirectly learning representation (through language model task)

**Predict ONE
Context Word
GIVEN ONE
target word**



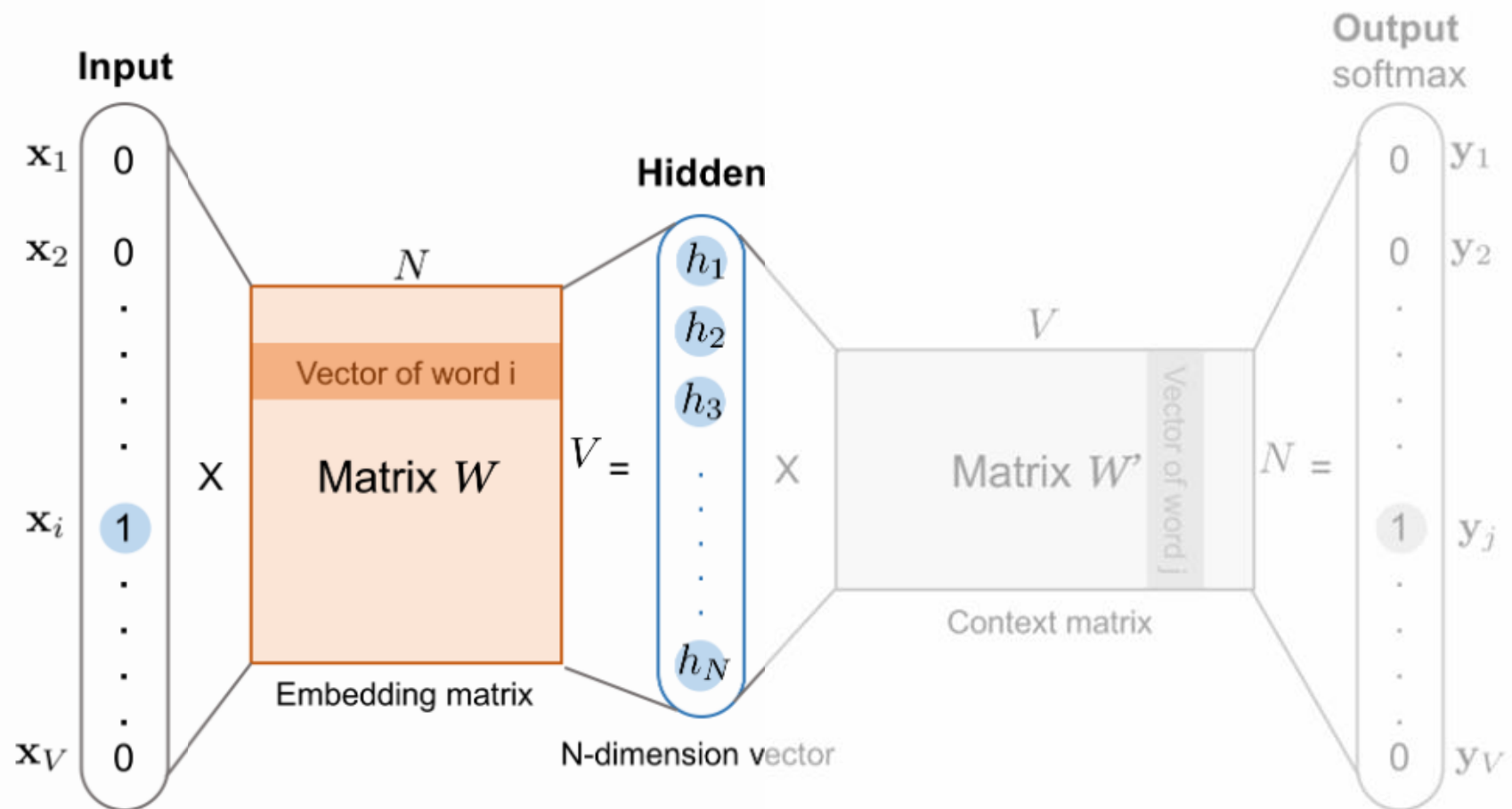
Word Representation: *Distributed Representation*

Word2Vec: Indirectly learning representation (through language model task)



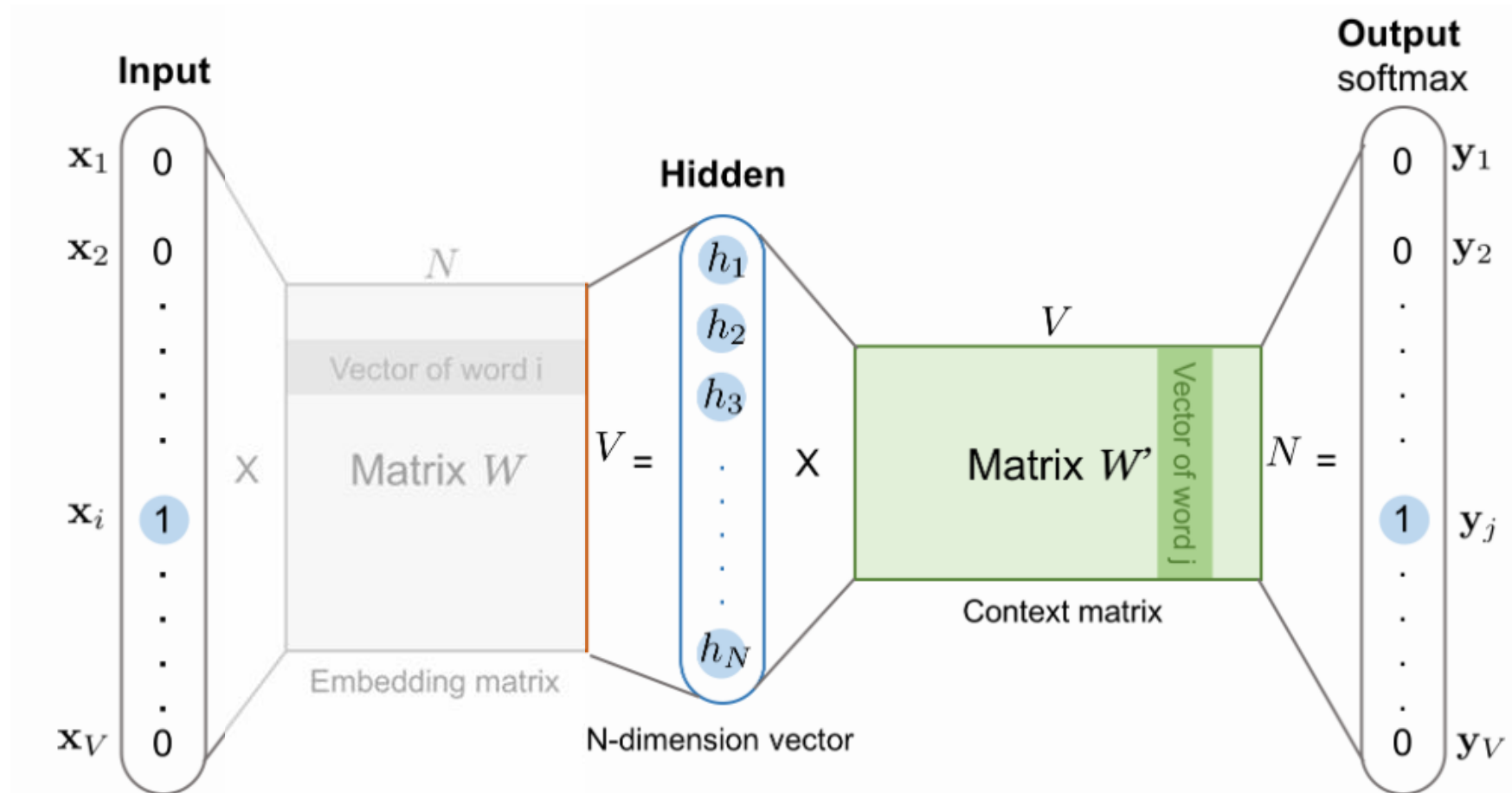
Word Representation: *Distributed Representation*

Word2Vec: Indirectly learning representation (through language model task)



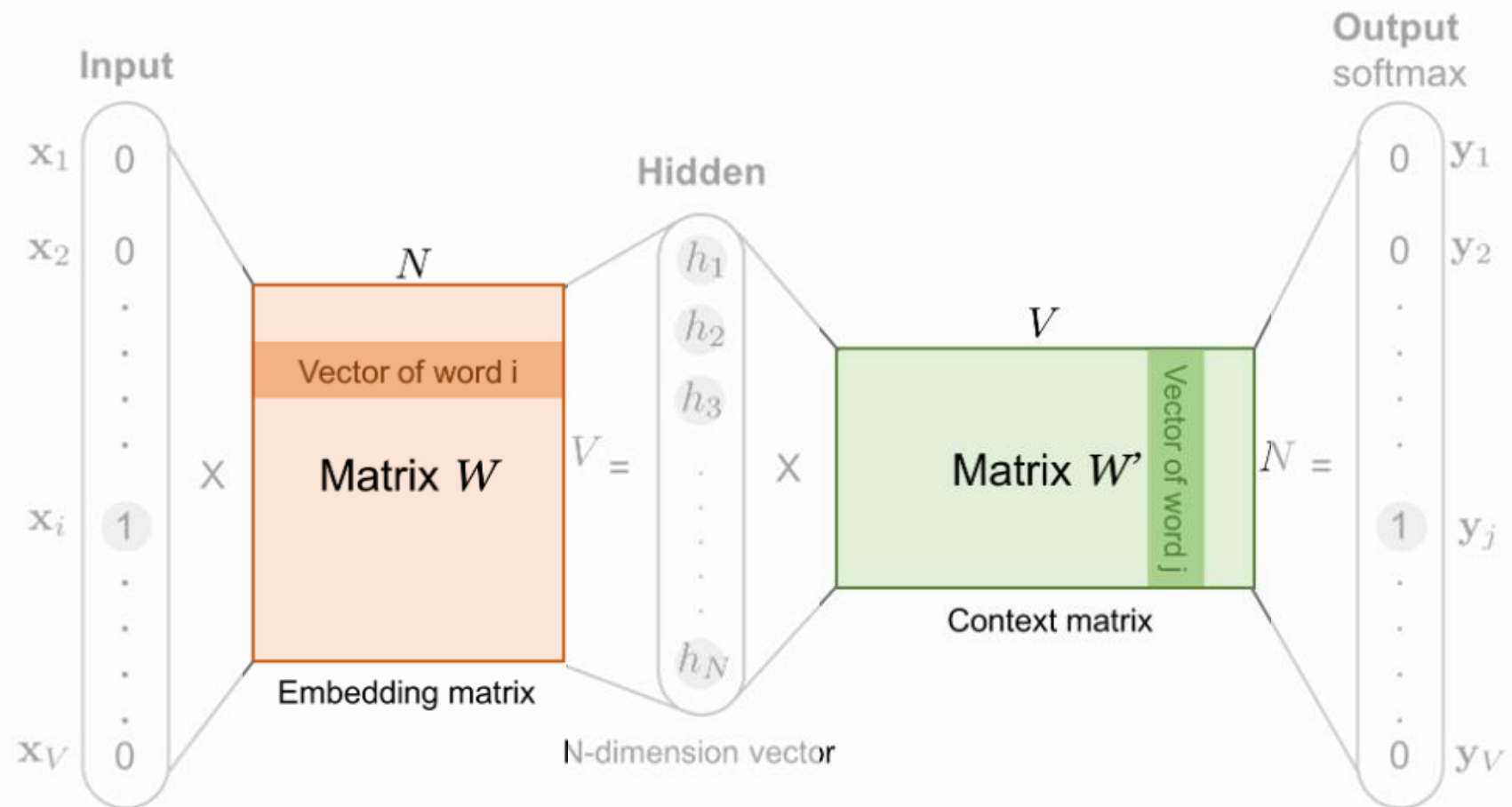
Word Representation: *Distributed Representation*

Word2Vec: Indirectly learning representation (through language model task)



Word Representation: *Distributed Representation*

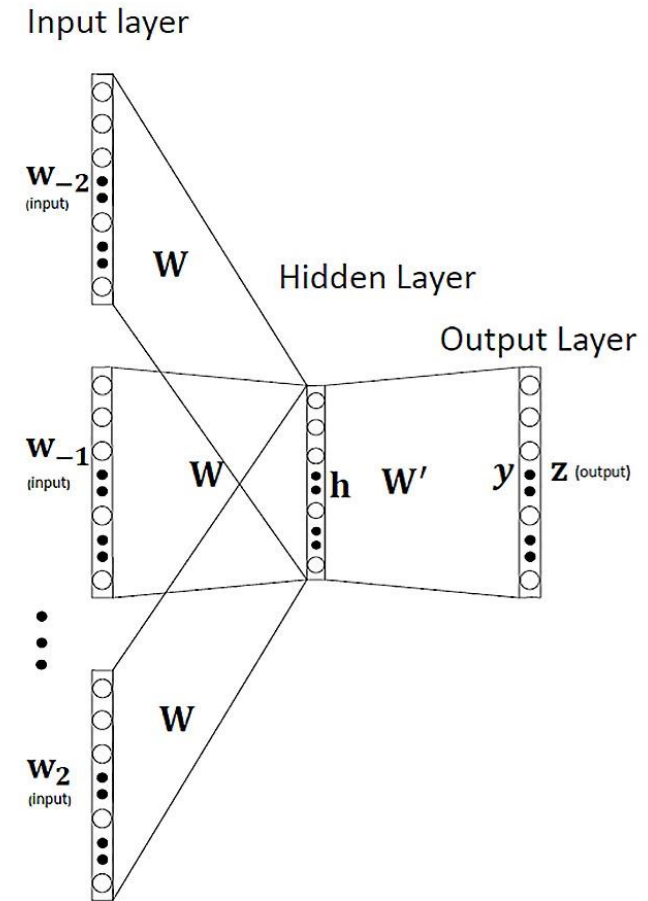
Word2Vec: Indirectly learning representation (through language model task)



Word Representation: *Distributed Representation*

Word2Vec: Continuous Bag-Of-Words (CBOW)

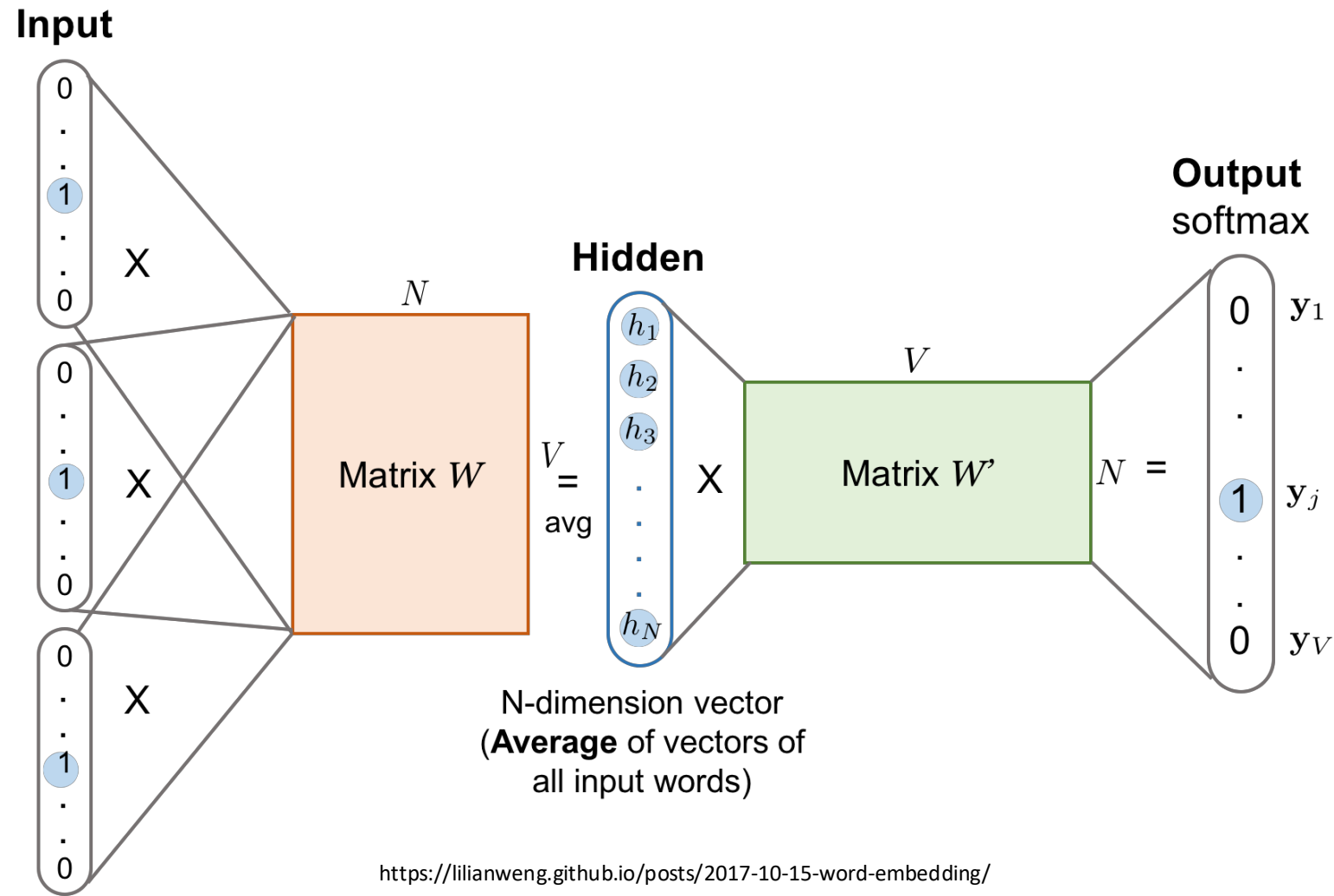
Given context, predict word (target)



(a) CBOW

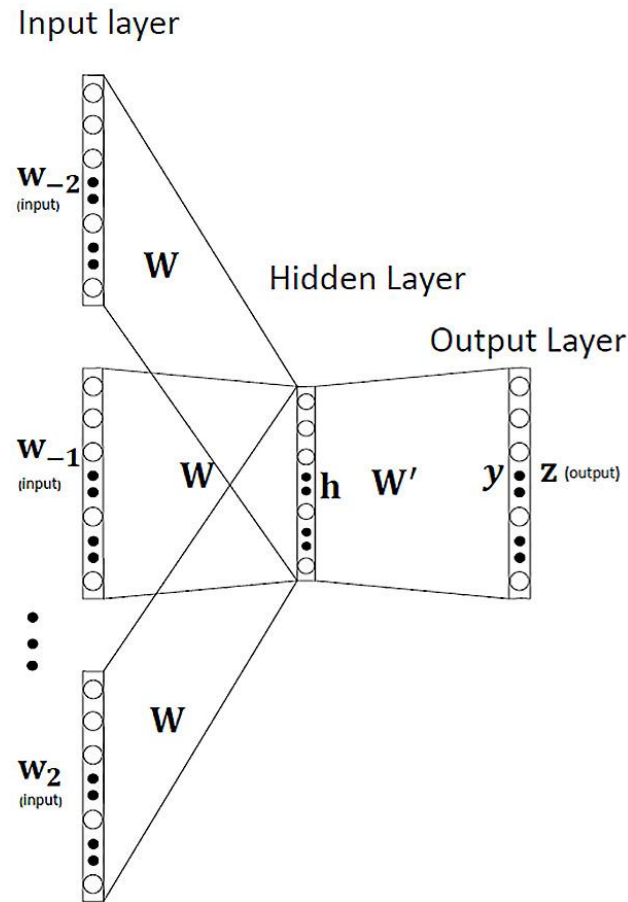
Word Representation: *Distributed Representation*

Word2Vec: Continuous Bag-Of-Words (CBOW)

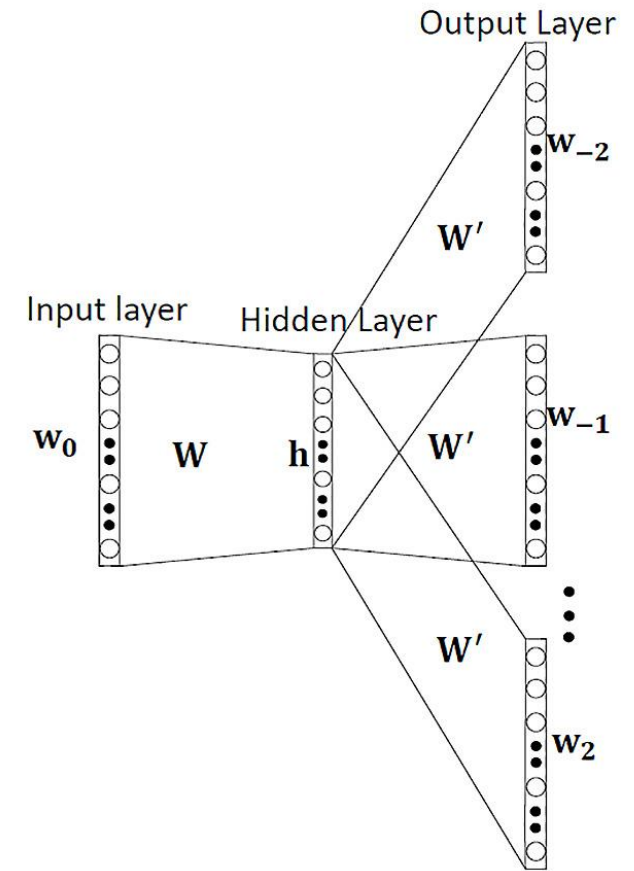


Word Representation: *Distributed Representation*

Word2Vec: CBOW vs. Skip-gram



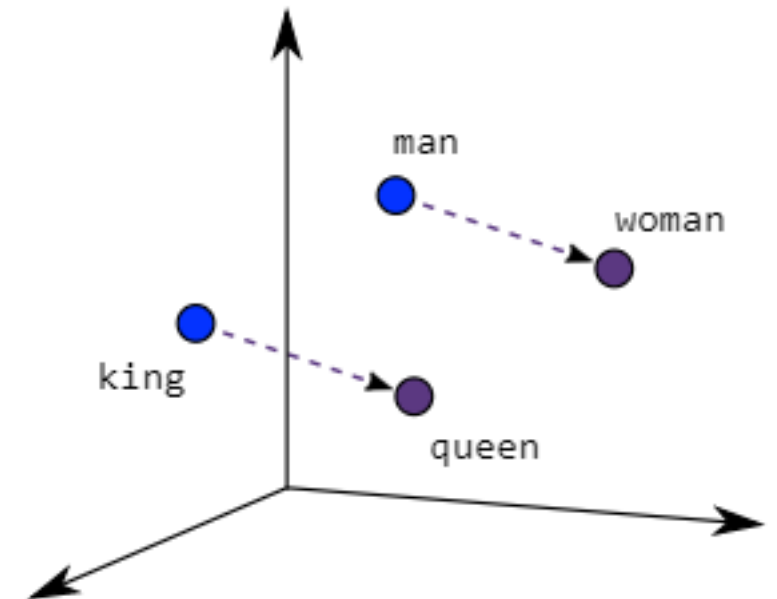
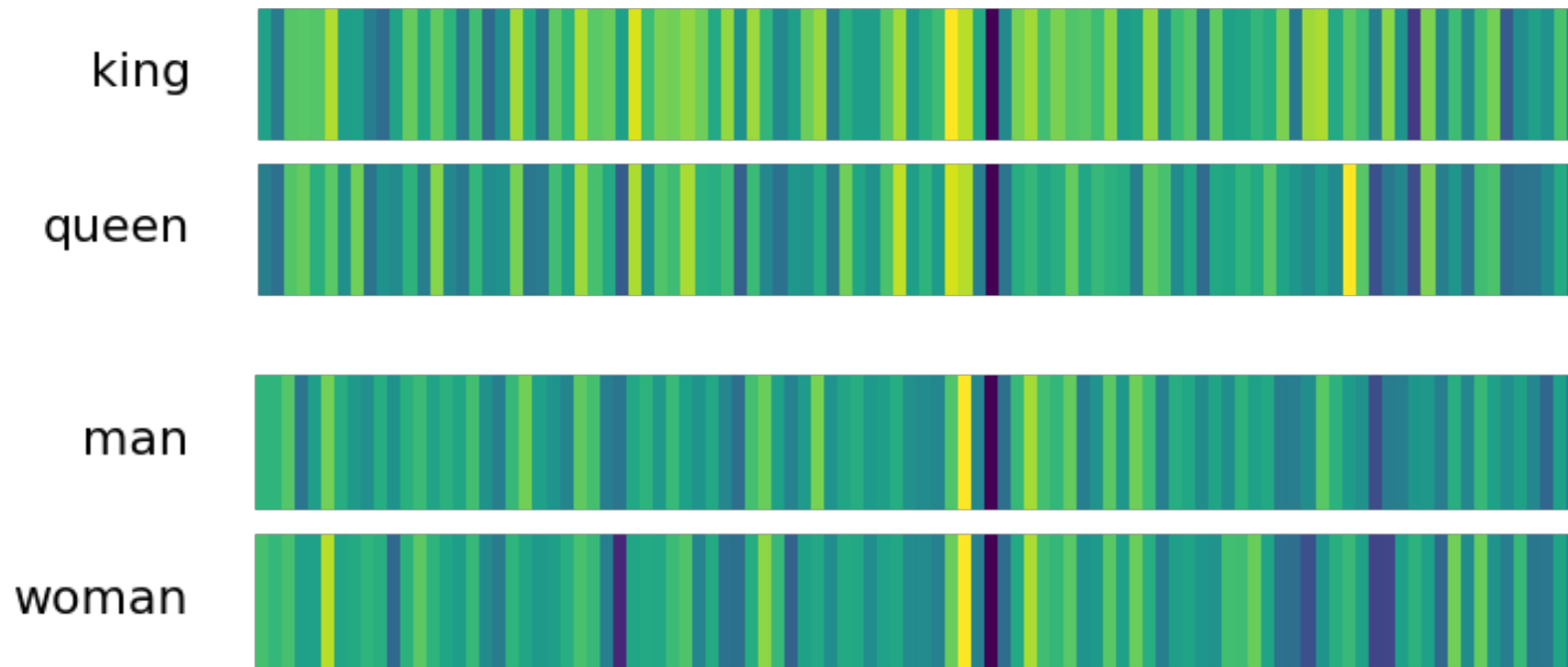
(a) CBOW



(b) Skip-Gram

Word Representation: *Distributed Representation*

Word2Vec: *Analogy*



Word Representation: *Distributed Representation*

Word2Vec: DEMO

<https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/index.html>

<https://remykarem.github.io/word2vec-demo/>

Embedding Projector: <https://projector.tensorflow.org/>

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- An unsupervised learning model developed by researchers at Stanford, 2014
- designed to effectively capture the meaning of words by analyzing their co-occurrence patterns in a large text corpus.
- core idea behind GloVe is to leverage both global statistics and local context of words.
- Why called Global? the statistics are captured at a global level

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- **Intuition:** ratios of word co-occurrence probabilities can encode some form of meaning

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

	Probability and Ratio					Context word
		$k = solid$	$k = gas$	$k = water$	$k = fashion$	
target word	$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}	
	$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}	

asking how strongly they co-occur with different context words

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \textit{solid}$	$k = \textit{gas}$	$k = \textit{water}$	$k = \textit{fashion}$
$P(k \textit{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \textit{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}

“How likely is context word k to appear near **ice**?”

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}

ice is more similar to *solid* than *steam* $\rightarrow P(\text{solid}|\text{ice}) > P(\text{solid}|\text{steam})$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \textit{solid}$	$k = \textit{gas}$	$k = \textit{water}$	$k = \textit{fashion}$
$P(k \textit{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \textit{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}

both *ice* and *steam* are NOT related to *fashion* → standalone probability values will not help us to infer the lack of relationship

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	<i>k = solid</i>	<i>k = gas</i>	<i>k = water</i>	<i>k = fashion</i>
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

$\gg 1$

Large ratio $\rightarrow$ strong semantic discrimination

Encodes:

ice is solid-like, steam is not

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

$\ll 1$

gas strongly favors **steam**

Encodes:

steam is gaseous, ice is not

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

≈ 1

water is relevant to **both ice and steam**

Encodes:

shared semantic ground (phase relationship)

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

≈ 1

fashion is unrelated to either word

Encodes:
irrelevant context $\rightarrow$ no semantic signal

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

both *ice* and *steam* are NOT related to *fashion* $\rightarrow \frac{P(\text{fashion}|\text{ice})}{P(\text{fashion}|\text{steam})} \approx 1$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	HIGH	LOW	HIGH	LOW
$P(k \text{steam})$	LOW	HIGH	HIGH	LOW
$P(k \text{ice})/P(k \text{steam})$	>1	<1	≈ 1	≈ 1

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

Training Process:

- **Initialization:** Word vectors are randomly initialized.
- **Optimization:** The model iteratively adjusts the word vectors to minimize the objective function. This is usually done using gradient descent.
- **Result:** The end result is a set of word vectors where each word is represented by a high-dimensional vector (typically a few hundred dimensions).

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

Goal: model a target function (such as F) so that “ratio of conditional probabilities represents the word meanings”

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

where

w_i, w_j : words in context

$\tilde{w}_k$: word out of context

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$F(\mathbf{w}_i, \mathbf{w}_j, \tilde{\mathbf{w}}_k) = \frac{P_{ik}}{P_{jk}}$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

vector scalar

The diagram illustrates the mathematical components of the GloVe function. The equation $F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$ is centered. Below it, the word 'vector' is positioned under the first three arguments of the function F , with an arrow pointing from the first argument w_i to the label. Similarly, the word 'scalar' is positioned under the two probabilities in the fraction, with an arrow pointing from the numerator P_{ik} to the label.

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$F\left((w_i - w_j)^T \cdot \tilde{w}_k\right) = \frac{P_{ik}}{P_{jk}}$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$F(w_i^T \cdot \tilde{w}_k - w_j^T \cdot \tilde{w}_k) = \frac{F(w_i^T \cdot \tilde{w}_k)}{F(w_j^T \cdot \tilde{w}_k)}$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$F(w_i^T \cdot \tilde{w}_k - w_j^T \cdot \tilde{w}_k) = \frac{F(w_i^T \cdot \tilde{w}_k)}{F(w_j^T \cdot \tilde{w}_k)}$$

$$F(a - b) = \frac{F(a)}{F(b)}$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

Applying Homomorphism

DEFINITION

Let A and B be groups, with operations given by $\circ_A$ and $\circ_B$ respectively. A **group homomorphism** $f: A \rightarrow B$ is a function f such that $f(x \circ_A y) = f(x) \circ_B f(y)$ for all $x, y \in A$.

$$F(w_i^T \cdot \tilde{w}_k - w_j^T \cdot \tilde{w}_k) = \frac{F(w_i^T \cdot \tilde{w}_k)}{F(w_j^T \cdot \tilde{w}_k)}$$

$$F(a - b) = \frac{F(a)}{F(b)}$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$F(w_i^T \cdot \tilde{w}_k - w_j^T \cdot \tilde{w}_k) = \frac{F(w_i^T \cdot \tilde{w}_k)}{F(w_j^T \cdot \tilde{w}_k)}$$

$$F(a - b) = \frac{F(a)}{F(b)} \quad \Rightarrow \quad F(x) = e^x$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$\begin{aligned} F((w_i - w_j)^T \cdot \tilde{w}_k) &= \frac{F(w_i^T \cdot \tilde{w}_k)}{F(w_j^T \cdot \tilde{w}_k)} \\ F(w_i, w_j, \tilde{w}_k) &= \frac{P_{ik}}{P_{jk}} \end{aligned} \quad \Rightarrow \quad \begin{aligned} F(w_i^T \cdot \tilde{w}_k) &= P_{ik} = \frac{X_{ik}}{X_i} \\ F(w_j^T \cdot \tilde{w}_k) &= P_{jk} = \frac{X_{jk}}{X_j} \end{aligned}$$

where

X_{ik} refers to the number of times word i appeared beside word k

X_i is the summation of the number of times all words appear in the context of word i

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$e^{(w_i^T \cdot \tilde{w}_k)} = P_{ik}$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$\log(e^{(w_i^T \cdot \tilde{w}_k)}) = \log(P_{ik})$$

$$w_i^T \cdot \tilde{w}_k = \log(X_{ik}) - \log(X_i)$$

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$w_i^T \cdot \tilde{w}_k = \log(X_{ik}) - \log(X_i)$$

$$J = \sum_{i,k=1}^V w_{ik} (w_i^T \cdot \tilde{w}_k - \log(X_{ik}) + \log(X_i))^2$$

V : vocab size

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

In simple words but mathematically:

$$w_i^T \cdot \tilde{w}_k = \log(X_{ik}) - \log(X_i)$$

$$J = \sum_{i,k=1}^V w_{ik} (w_i^T \cdot \tilde{w}_k - \log(X_{ik}) + \log(X_i))^2$$

b_i

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- 1) **Co-occurrence Matrix:** build a co-occurrence matrix, where each cell (i, j) represents the number of times word i and word j appear together within a specific context window.
- 2) Weighting Function
- 3) Objective Function
- 4) Optimization
- 5) Resulting Word Embeddings

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- 1) Co-occurrence Matrix
- 2) **Weighting Function:** apply a weighting function to the co-occurrence counts to emphasize important co-occurrences and de-emphasize less important ones. The weighting function typically considers factors like word frequency and context window size.
- 3) Objective Function
- 4) Optimization
- 5) Resulting Word Embeddings

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- 1) Co-occurrence Matrix
- 2) Weighting Function
- 3) **Objective Function:** minimizes an objective function based on the ratio of weighted co-occurrence counts and the dot product of the corresponding word vectors.

$$J(\theta) = \sum_i \sum_j \left(f(X_{ij}) - \text{dot}(v_i, v_j) \right)^2$$

θ represents the set of all word vectors

v_i and v_j are the word vectors for words i and j , respectively.

- 4) Optimization
- 5) Resulting Word Embeddings

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- 1) Co-occurrence Matrix
- 2) Weighting Function
- 3) Objective Function
- 4) **Optimization:** uses gradient-based optimization techniques like Stochastic Gradient Descent (SGD) to minimize the objective function. This process iteratively updates the word vectors to achieve better alignment with the co-occurrence patterns in the text.
- 5) Resulting Word Embeddings

Word Representation: *Distributed Representation*

Global Vectors for Word Representation (GloVe) <https://nlp.stanford.edu/pubs/glove.pdf>

- 1) Co-occurrence Matrix
- 2) Weighting Function
- 3) Objective Function
- 4) Optimization
- 5) **Resulting Word Embeddings:** After optimization, GloVe outputs a set of word vectors that capture the semantic meaning and relationships between words.

Word Representation: *Distributed Representation*

Limitations

- **Context-Agnostic:** assign the same vector to a word, regardless of its context

mouse¹ : a *mouse* controlling a computer system in 1968.

mouse² : a quiet animal like a *mouse*

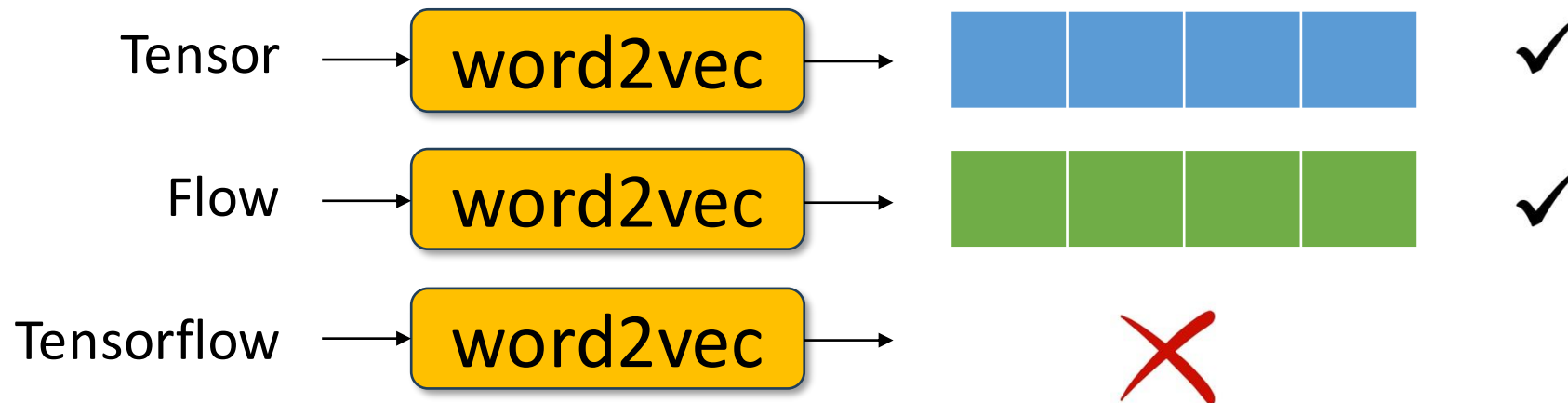
bank¹ : ...a *bank* can hold the investments in a custodial account ...

bank² : ...as agriculture burgeons on the east *bank*, the river ...

Word Representation: *Distributed Representation*

Limitations

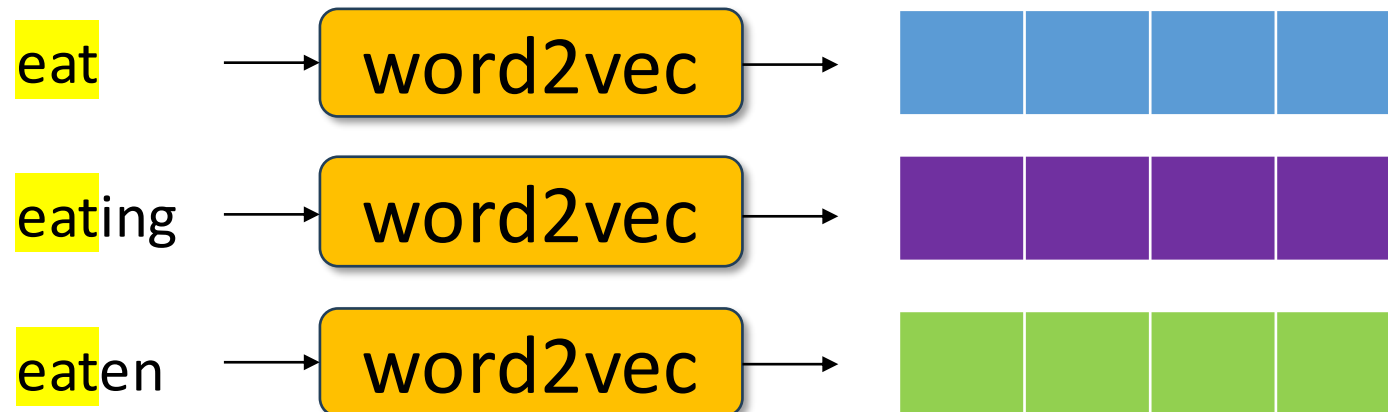
- Context-Agnostic
- **Handling Out-of-Vocabulary (OOV) Words:** If a word wasn't seen during training, it won't have an embedding.



Word Representation: *Distributed Representation*

Limitations

- Context-Agnostic
- Handling Out-of-Vocabulary (OOV) Words:
- **Morphology:** For words with same radicals such as “eat” and “eaten”, Word2Vec doesn’t do any parameter sharing. Each word is learned uniquely based on the context it appears in



Word Representation: *Distributed Representation*

FastText

<https://arxiv.org/pdf/1607.04606.pdf>

- Proposed by Facebook AI Research (FAIR), 2016
- An extension of the word2vec model
- To improve vector representation for morphologically rich language
- Provides embeddings for character n-grams, representing words as the average of these embeddings

Word Representation: *Distributed Representation*

FastText

Example:

- “equal” and $n = 3$, then the word will be represented by character n-grams:

< eq, equ, qua, ual, al > and < equal >

The word embedding for the word ‘equal’ can be given as the sum of all vector representations of all of its character n-gram and the word itself.

Word Representation: *Distributed Representation*

Limitations

- Context-Agnostic
- Handling Out-of-Vocabulary (OOV) Words
- Morphology
- **Dependency on Training Data:** The quality and breadth of the embeddings depend heavily on the training corpus.

Word Representation: *Distributed Representation*

Limitations

- Context-Agnostic
- Handling Out-of-Vocabulary (OOV) Words
- Morphology
- **Dependency on Training Data:** The quality and breadth of the embeddings depend heavily on the training corpus.
 - **Domain Specificity:** Embeddings trained on general corpora may not perform well on specialized or domain-specific tasks without further fine-tuning or domain-specific training data.

Word Representation: *Distributed Representation*

Limitations

- Context-Agnostic
- Handling Out-of-Vocabulary (OOV) Words
- Morphology
- Dependency on Training Data
- **Dimensionality:** The embeddings are high-dimensional (usually in hundreds), which can be computationally expensive and challenging to manage.

Word Representation: *Distributed Representation*

Limitations

- Context-Agnostic
- Handling Out-of-Vocabulary (OOV) Words
- Morphology
- Dependency on Training Data
- Dimensionality
- **Interpretability:** The dense nature of these embeddings makes them hard to interpret compared to sparse representations like one-hot encoding.

Word Representation: *Contextualized Representation*

The word use (e.g., syntax and semantics) depends on its context.

mouse¹ : a *mouse* controlling a computer system in 1968.

mouse² : a quiet animal like a *mouse*

bank¹ : ...a *bank* can hold the investments in a custodial account ...

bank² : ...as agriculture burgeons on the east *bank*, the river ...

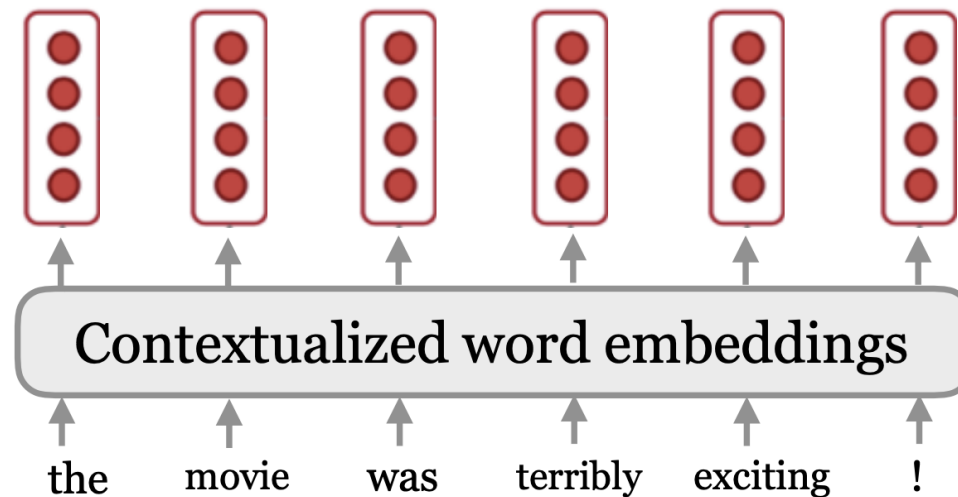
Word Representation: *Contextualized Representation*

IDEA: Build a vector for each word conditioned on its **context**!

Word Representation: *Contextualized Representation*

IDEA: Build a vector for each word conditioned on its **context**!

representation for each token is a function of the entire input sentence



$$g : (w_1, w_2, \dots, w_n) \longrightarrow \mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$$

Word Representation: *Contextualized Representation*

$f(\text{play} \mid \text{Elmo and Cookie Monster play a game})$

$\neq$

$f(\text{play} \mid \text{The Broadway play premiered yesterday})$

Reading: (Peters et al, 2018): Deep contextualized word representations

Word Representation: *Other Representation*

- **Multi-prototype Word Representation**
 - Instead of representing a word with a single embedding, multi-prototype models assign multiple vectors to a single word, each representing a different sense or usage of the word.
- Multisource Word Representation
- Multilingual Word Representation
- Task-Specific Word Representation
- Time-Specific Word Representation

Word Representation: *Other Representation*

- Multi-prototype Word Representation
- **Multisource Word Representation**
 - aims to create richer and more comprehensive word representations by incorporating information from multiple sources. Unlike traditional word embedding methods, which typically rely solely on text data, MultisourceWR leverages diverse sources of information to capture a more holistic understanding of word meaning.
 - Knowledge bases (WordNet), Lexical resources (dictionaries), Visual information (image/video), Audio data (Sound recordings and speech)
- Multilingual Word Representation
- Task-Specific Word Representation
- Time-Specific Word Representation

Word Representation: *Other Representation*

- Multi-prototype Word Representation
- Multisource Word Representation
- **Multilingual Word Representation**
 - aim to represent words across multiple languages in a unified and meaningful way.
- Task-Specific Word Representation
- Time-Specific Word Representation

Word Representation: *Other Representation*

- Multi-prototype Word Representation
- Multisource Word Representation
- Multilingual Word Representation
- **Task-Specific Word Representation**
 - aims to tailor word representations to the specific needs of a particular NLP task.
- Time-Specific Word Representation

Word Representation: *Other Representation*

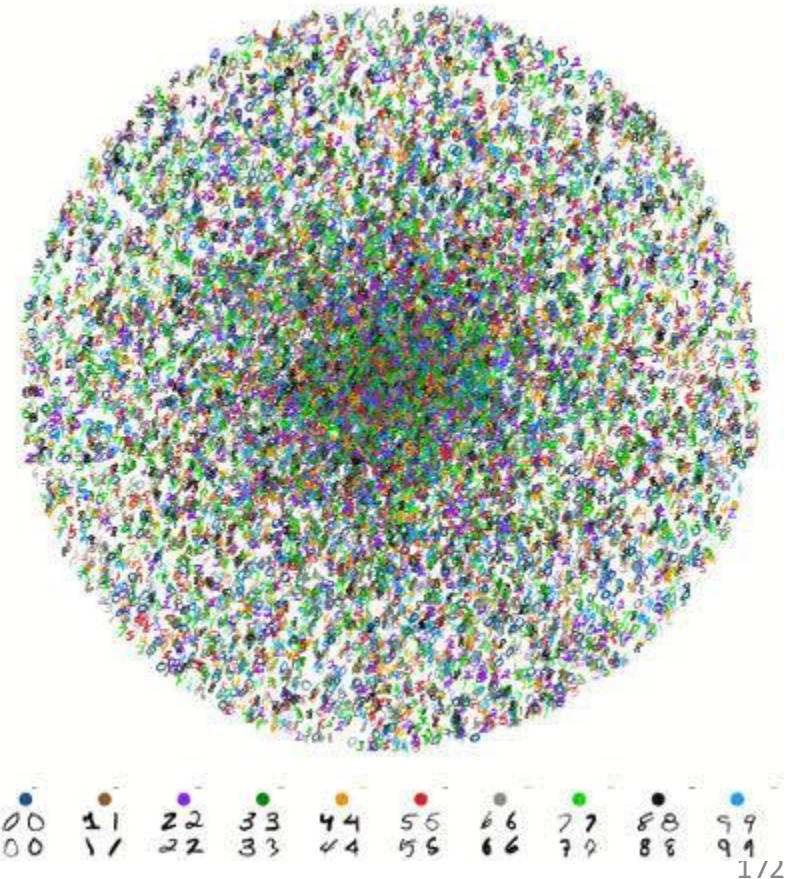
- Multi-prototype Word Representation
- Multisource Word Representation
- Multilingual Word Representation
- Task-Specific Word Representation
- **Time-Specific Word Representation**
 - aims to capture how the meaning and usage of words evolve over time.

Word Representation: *Visualization*

t-distributed Stochastic Neighbor Embedding (t-SNE)

- A powerful technique in dimensionality reduction used for visualizing high-dimensional data in a low-dimensional space, typically 2D or 3D.
- LIVE DEMO: <https://nicola17.github.io/tfjs-tsne-demo/>

Real-time evolution of the tSNE embedding for the complete MNIST dataset with our technique. The dataset contains images of 60,000 handwritten digits.



Word Representation: *Visualization*

t-distributed Stochastic Neighbor Embedding (t-SNE)

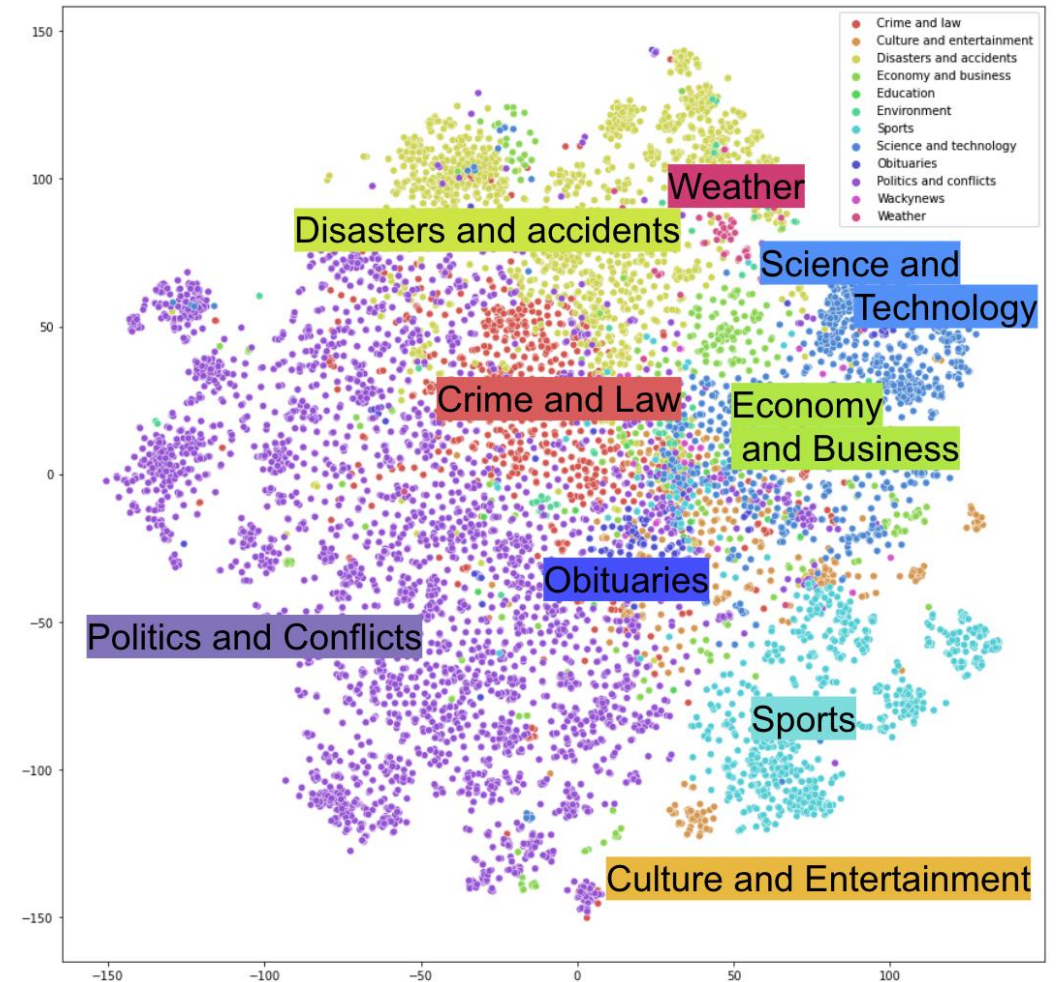
- A powerful technique in dimensionality reduction used for visualizing high-dimensional data in a low-dimensional space, typically 2D or 3D.

How it works:

1. **High-dimensional space:** The data points are initially represented in a high-dimensional space.
2. **Similarity measure:** A similarity measure is used to determine the pairwise similarities between data points (based on Euclidean distance, cosine similarity, or other metrics).
3. **Low-dimensional space:** A low-dimensional space is created, typically 2D or 3D.
4. **Stochastic embedding:** Data points are stochastically embedded into the low-dimensional space while preserving the pairwise similarities calculated in the high-dimensional space.
5. **Visualization:** Finally, the embedded data points are visualized in the low-dimensional space, revealing the relationships between them.

Sentence Representation

IDEA: transforming a sentence, a series of words strung together, into a numerical vector that encapsulates its meaning and semantic information.



Visualization of the Distribution of Sentence Embeddings By News Topics

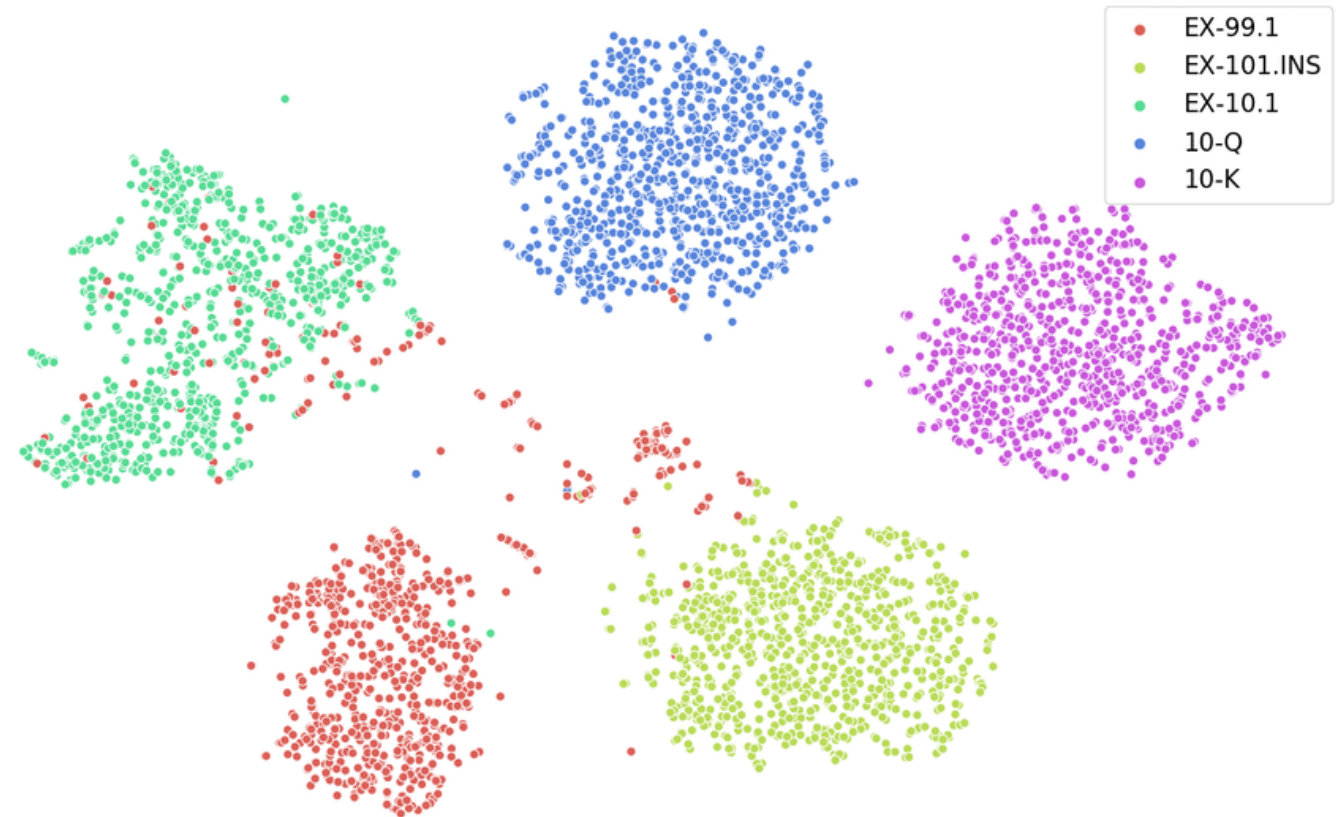
Sentence Representation

Techniques

- **Bag-of-Words (BoW):** Represents a sentence by a vector indicating the presence or frequency of words, disregarding grammar and word order.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** Similar to BoW but weighs the word frequencies, giving importance to words that are unique to a particular document.
- **Averaging Word Embeddings:** Involves averaging the embeddings (like Word2Vec or GloVe vectors) of all words in the sentence.
- **Neural Network:**
 - **Recurrent Neural Networks (RNNs):** process sentences word by word, capturing the sequential nature of language.
 - **Transformer Models:** generating rich contextualized sentence representations by considering the entire sentence and its context.
 - **Dedicated Sentence Embedding Models:** Models specifically designed for sentence embeddings, like the Universal Sentence Encoder or Sentence-BERT, provide embeddings optimized for capturing sentence-level semantics.

Document Representation

IDEA: transforming a document into a numerical format that computers can understand and analyze.



t-SNE plot of projections of document embeddings, using vanilla Doc2Vec.

<https://arxiv.org/pdf/1912.06905.pdf>

Document Representation

Techniques

- **Bag-of-Words (BoW):** Represents a document by a vector indicating the presence or frequency of words, disregarding grammar and word order.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** Similar to BoW but adjusts the word frequencies based on their commonness across documents, giving more weight to rare but potentially more significant words.
- **Topic Models** (like LDA - Latent Dirichlet Allocation): Identify topics present in a corpus and represent documents as a mixture of these topics. Each topic is characterized by a distribution over words.
- **Doc2Vec:** Similar to word2vec, generate fixed-length dense vector representations for entire documents.
- **Neural Network:**
 - **Transformer Models:** Models like BERT can be adapted for document representation by processing the text and generating an aggregate representation,

Word Representation: *in Python*

Scikit-learn (TF-IDF Vectorizer)

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = ['This is the first document.', 'This document is the second document.']

vectorizer = TfidfVectorizer() X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Word Representation: *in Python*

Gensim

```
from gensim.models import Word2Vec

# Load pre-trained model
model = Word2Vec.load("word2vec_model.bin")

# Get word embedding
word_embedding = model.wv["word"]

# Get similar words
similar_words = model.wv.most_similar("word")
```

Word Representation: *in Python*

spaCy

```
import spacy

# Load spaCy model
nlp = spacy.load("en_core_web_sm")

# Get word embedding
word_embedding = nlp("word").vector

# Get document embedding
document_embedding = nlp("This is a sentence").vector
```

Word Representation: *in Python*

fastText

```
from fasttext import load_model

# Load pre-trained model
model = load_model("fasttext_model.bin")

# Get word embedding
word_embedding = model.get_word_vector("word")

# Get similar words
similar_words = model.get_nearest_neighbors("word")
```

Word Representation: *in Python*

Context2Vec

```
from context2vec import C2V

# Load pre-trained model
model = C2V.load("context2vec_model.bin")

# Get word embedding
word_embedding = model.get_word_embedding("word", "This is the context sentence.")

# Get similar words
similar_words = model.most_similar("word", "This is the context sentence.")
```

Word Representation: *in Python*

Training Word Embedding Models

- PyTorch
- Keras
- Tensorflow
- Hugging Face's Transformers

```
import tensorflow as tf

# Define model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_len),
    tf.keras.layers.Conv1D(filters=128, kernel_size=3),
    tf.keras.layers.GlobalMaxPooling1D(),
    tf.keras.layers.Dense(embedding_dim)
])

# Train model
model.compile(loss="mse", optimizer="adam")
model.fit(X_train, Y_train)

# Get word embedding
word_embedding = model.predict(np.array(["word"]))
```

References

Readings:

- Bengio et al., 2003, [A Neural Probabilistic Language Model](#)
- Mikolov et al., 2013, [Efficient Estimation of Word Representations in Vector Space](#)
- Pennington et al., 2014, [GloVe: Global Vectors for Word Representation](#)
- Bojanowski et al., 2017, [Enriching Word Vectors with Subword Information](#)

Tutorials:

- Word2vec: <https://jalammr.github.io/illustrated-word2vec/>
- Word Embedding: <https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/tutorial.htm> |

Fun:

- <https://research.google.com/semantris/>